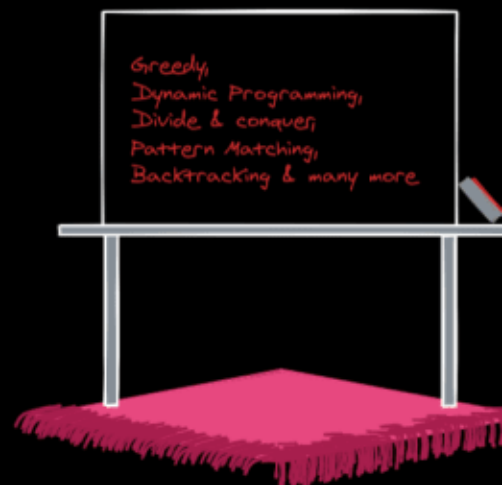
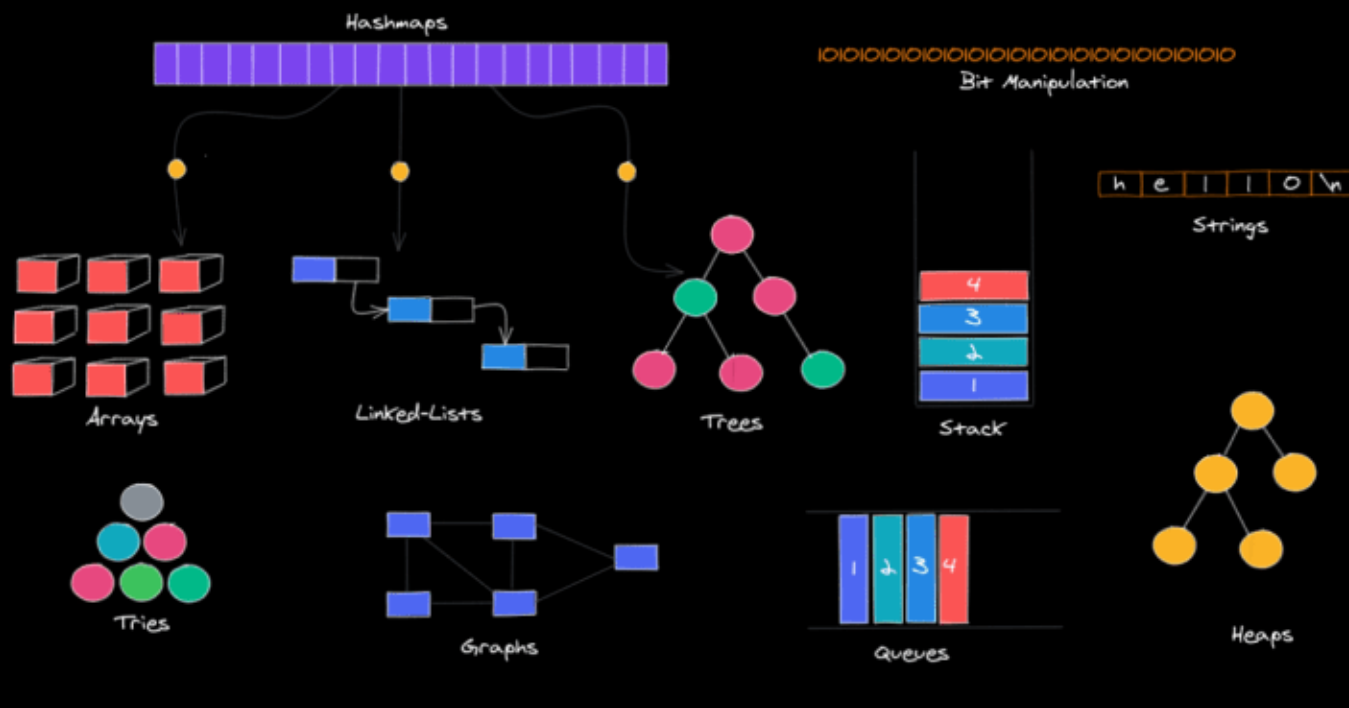




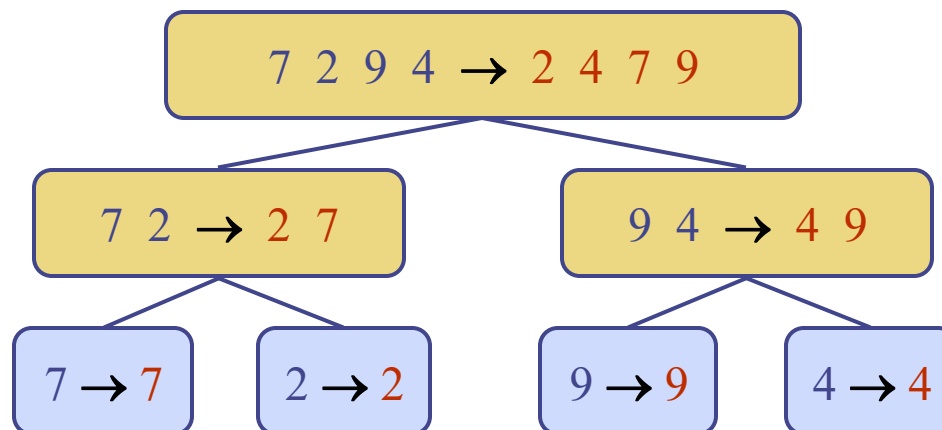
数据结构与算法导论

刘子博

FDSM

liuzibo@fudan.edu.cn

排序



排序

- **排序 (sorting)** 算法是一类重要算法，已被众多学者充分研究过，且在实践中有重大价值
 - 许多解决不同问题的算法均依赖于排序算法的实现
 - 数据分析与处理中经常会用到排序算法
 -
- Python对排序提供list类的sort及对任意元素集合的sorted函数。这些函数使用高度优化后的高级排序算法。学习排序算法后可以对这些函数的效率有直观的认知

排序

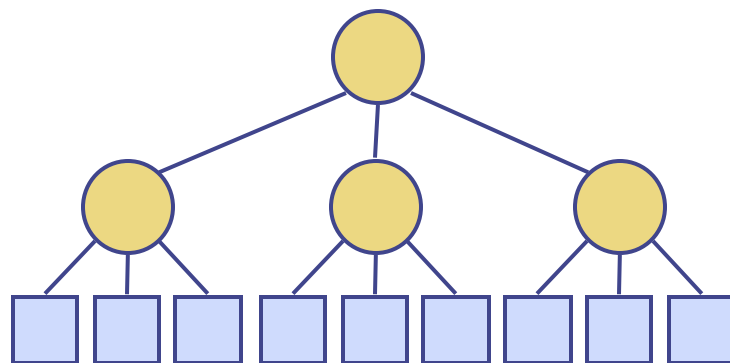
- 我们已经了解过的排序算法：
 - 插入排序 ($O(n^2)$)
 - 选择排序 ($O(n^2)$)
 - 堆排序 ($O(n \log n)$)
- 另一种常见的简单排序方法：
 - 冒泡排序 (bubble-sort)

冒泡排序

- 冒泡排序：
 - 假设序列为 $a_1, a_2, \dots, a_n$
 - 第一次扫描：从序列尾向序列头进行，比较相邻两个元素的大小，如出现逆序 ($a_j > a_{j+1}$) 则交换两个元素
 - ◆ 第一次扫描结束后，最小的元素被排在第一位
 - 第二次扫描：依然从序列尾向序列头进行，出现逆序则交换相邻元素，一直进行到第二个元素为止
 -
 - 冒泡排序的名字来源于其算法的过程
 - 时间复杂度？

分治法

- 我们接下来描述的两个算法均由分治法思想产生
- 分治法 (Divide-and-conquer) 是一种通用的算法设计范式:
 - 分解 (Divide) : 如输入数据 S 规模较小则直接解决之; 否则, 将其分成两个或多个互斥子集 $S_1, S_2, \dots$
 - 解决子问题 (Conquer) : 递归解决与子集相关的子问题
 - 合并 (Combine) : 将子问题 $S_1, S_2, \dots$ 的解合并为 S 的解



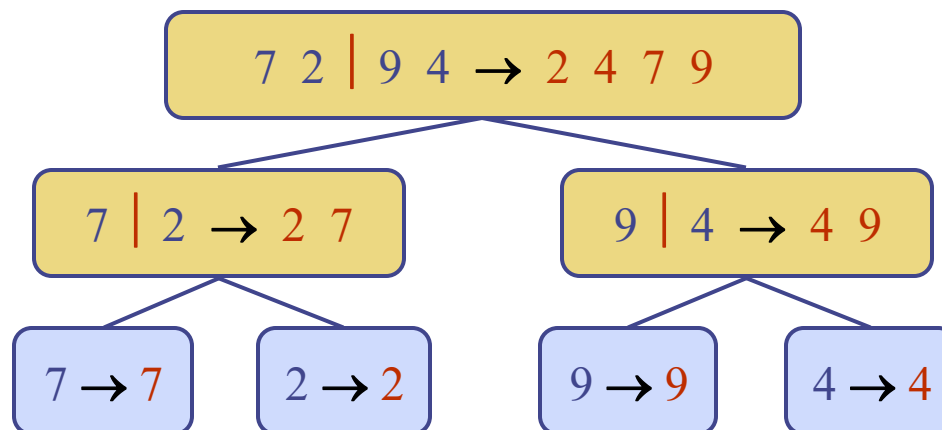
排序

- 归并排序
- 快速排序
- 排序下界
- 桶排序和基数排序

排序

- 归并排序
- 快速排序
- 排序下界
- 桶排序和基数排序

归并排序



归并排序

- 归并排序 (merge-sort) 基于分治法思想。在有 n 个元素的序列 S 上执行归并排序的过程如下：
 - 分解 (Divide) : 当序列 S 长度大于1时, 将其分为两个长度大致为 $n/2$ 的子序列 S_1 和 S_2
 - 解决子问题 (Conquer) : 递归地对 S_1 和 S_2 进行排序
 - 合并 (Combine) : 将排好序的 S_1 和 S_2 合并为一个有序的序列

Algorithm *mergeSort*(S)

Input sequence S with n elements

Output sequence S sorted

if $S.size() > 1$

$(S_1, S_2) \leftarrow partition(S, n/2)$

mergeSort(S_1)

mergeSort(S_2)

$S \leftarrow merge(S_1, S_2)$

合并两个有序序列

- 在归并排序的最后一步（合并）中，需要将两个有序序列 A 和 B 合并为一个有序序列 S
- 这一合并操作的时间复杂度为 $O(n)$

Algorithm *merge*(A, B)

Input sequences A and B with $n/2$ elements each

Output sorted sequence of $A \cup B$

$S \leftarrow$ empty sequence

while $\neg A.isEmpty() \wedge \neg B.isEmpty()$

if $A.first().element() < B.first().element()$

$S.addLast(A.remove(A.first()))$

else

$S.addLast(B.remove(B.first()))$

while $\neg A.isEmpty()$

$S.addLast(A.remove(A.first()))$

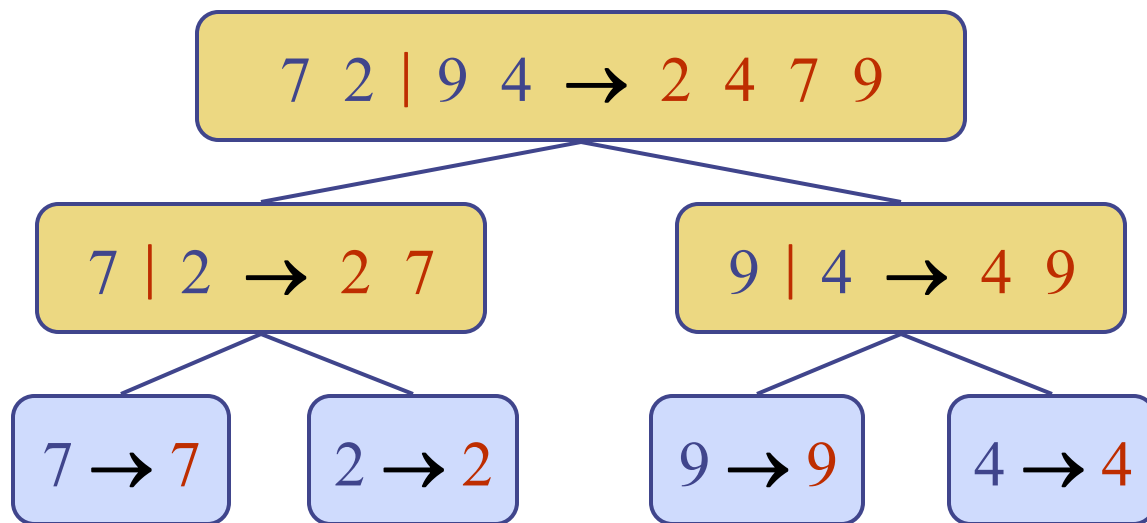
while $\neg B.isEmpty()$

$S.addLast(B.remove(B.first()))$

return S

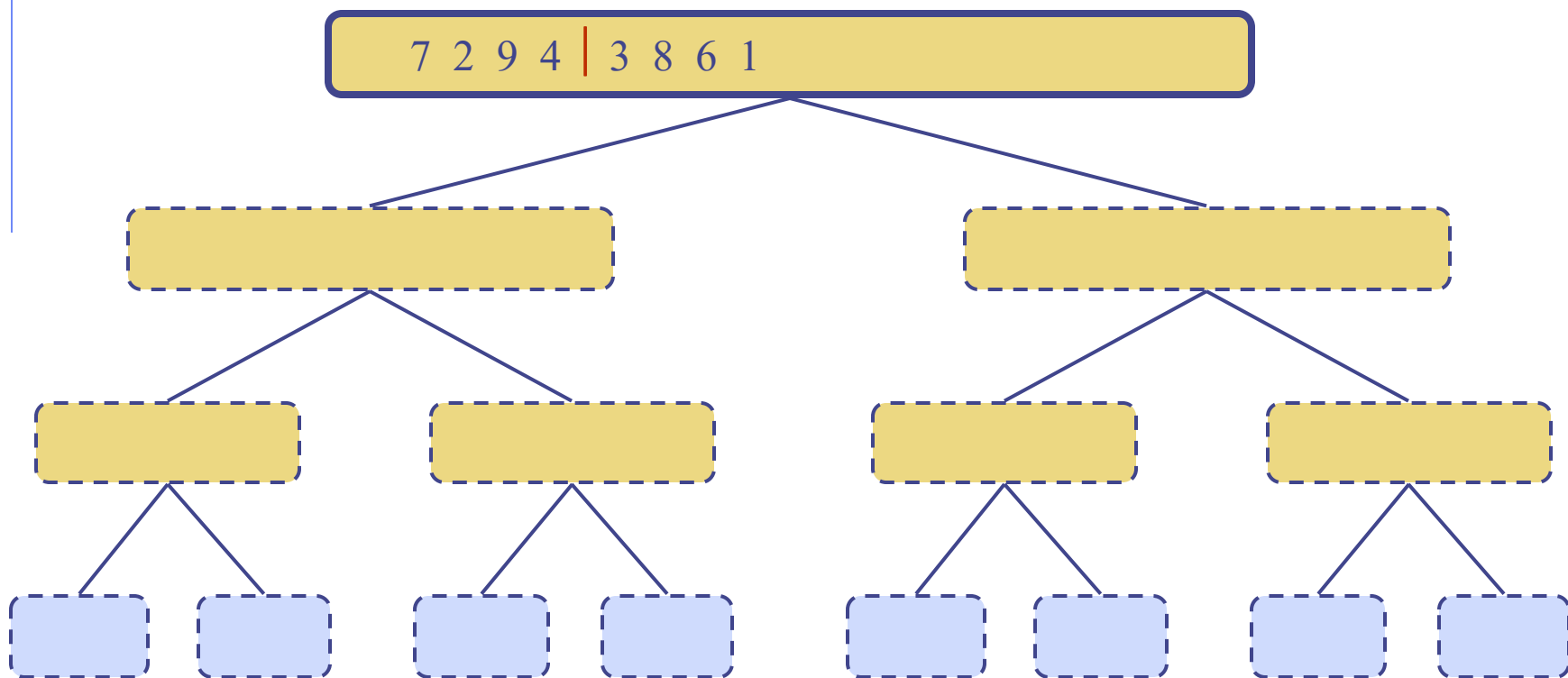
归并排序树

- 归并排序的执行过程可以用一棵二叉树很好地表示，我们称之为**归并排序树** (merge-sort tree)
 - 归并排序树的每一个节点代表一次递归调用
 - 根节点是首次调用归并排序的位置
 - 叶子节点为对长为0或1对序列进行排序



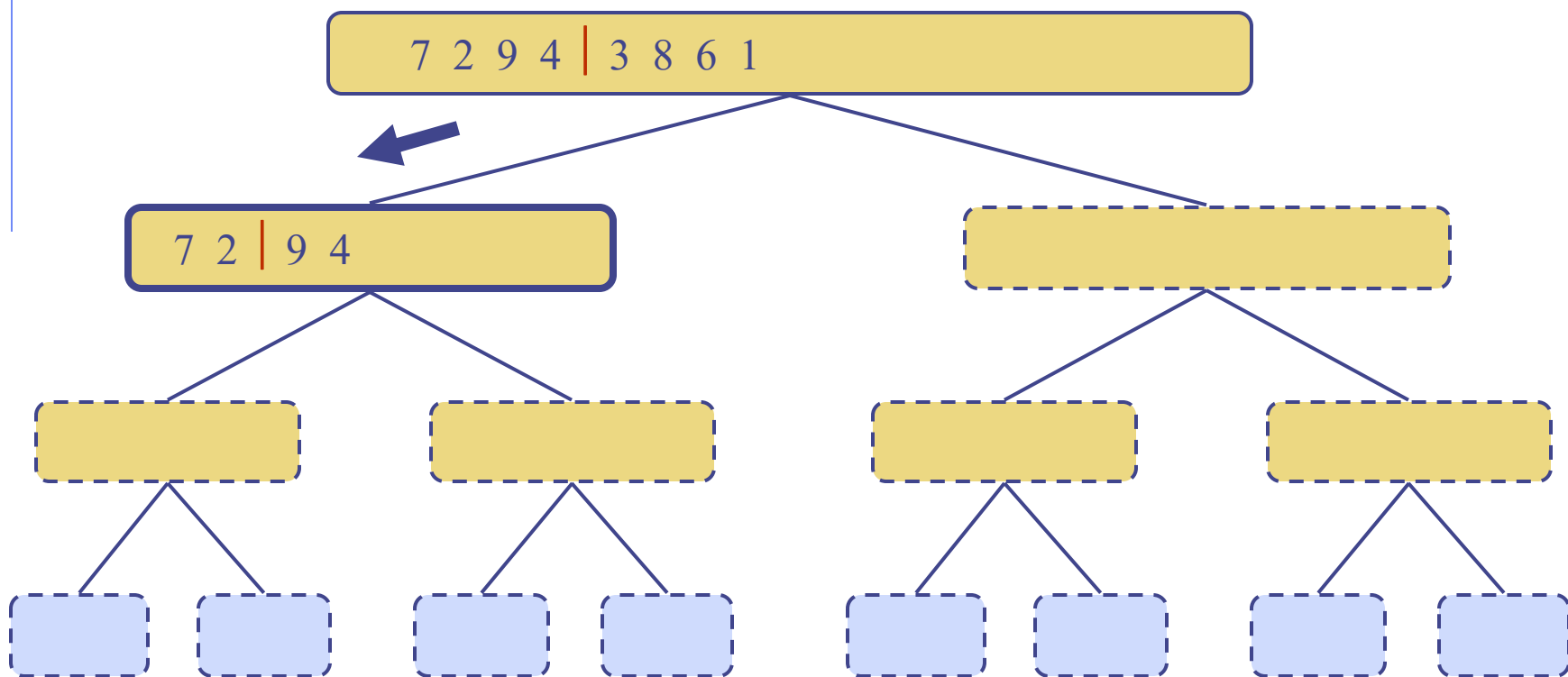
归并排序例

- 分解



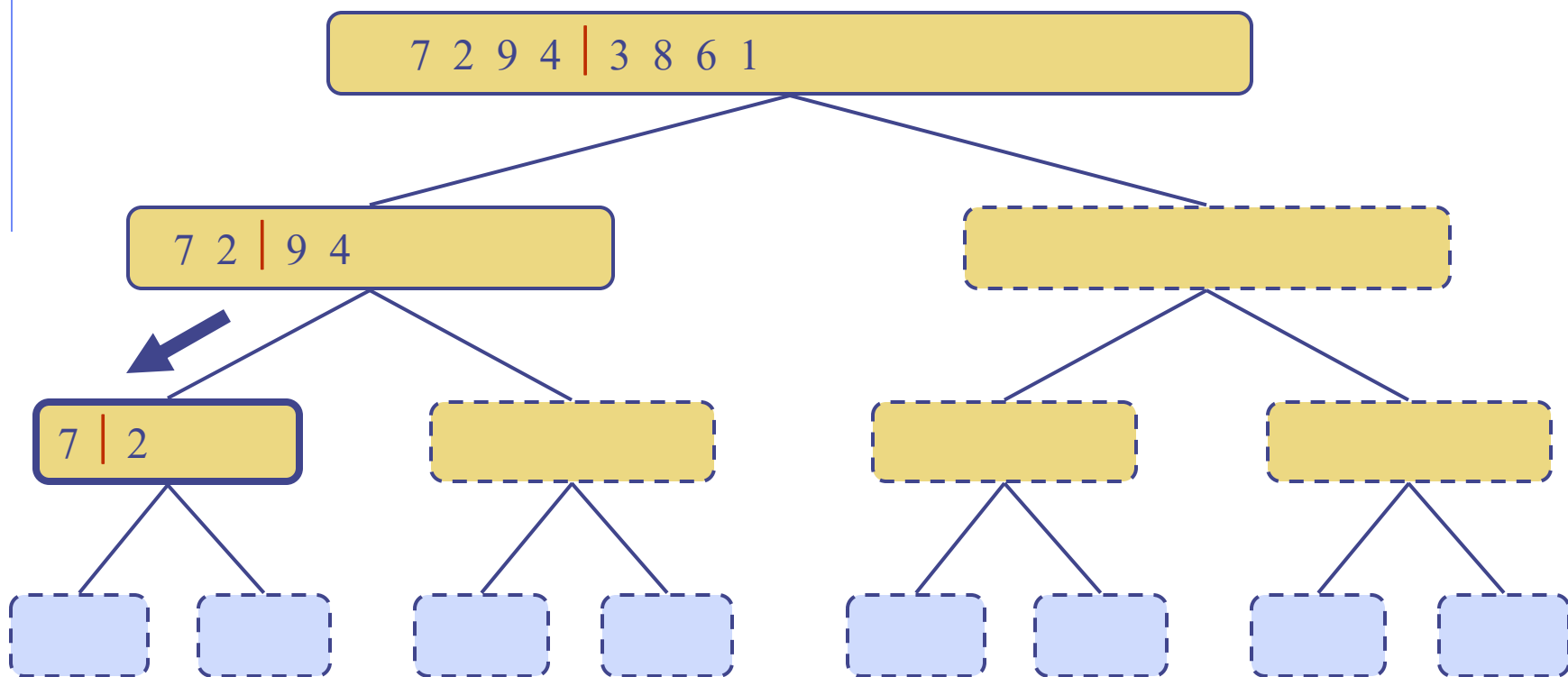
归并排序例

- 递归调用，继续分解



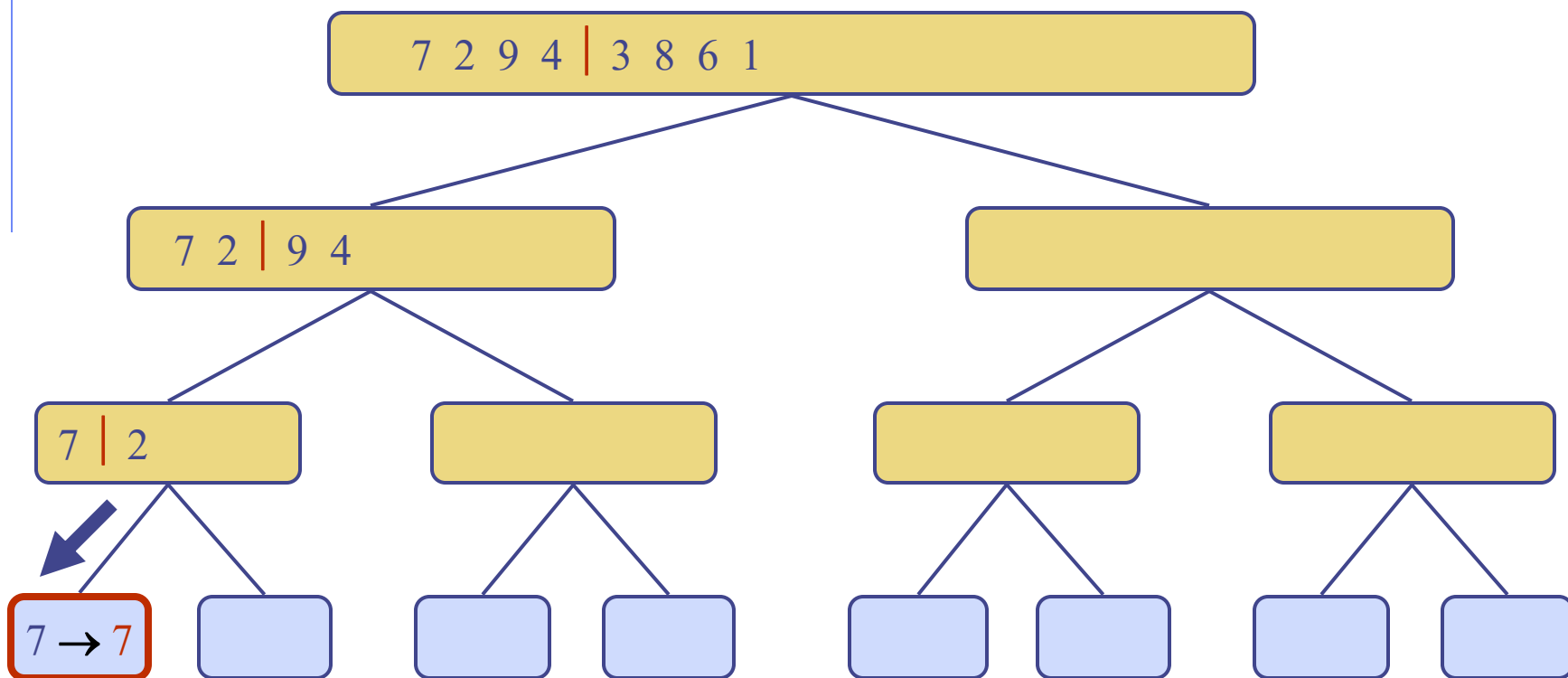
归并排序例

- 递归调用，继续分解



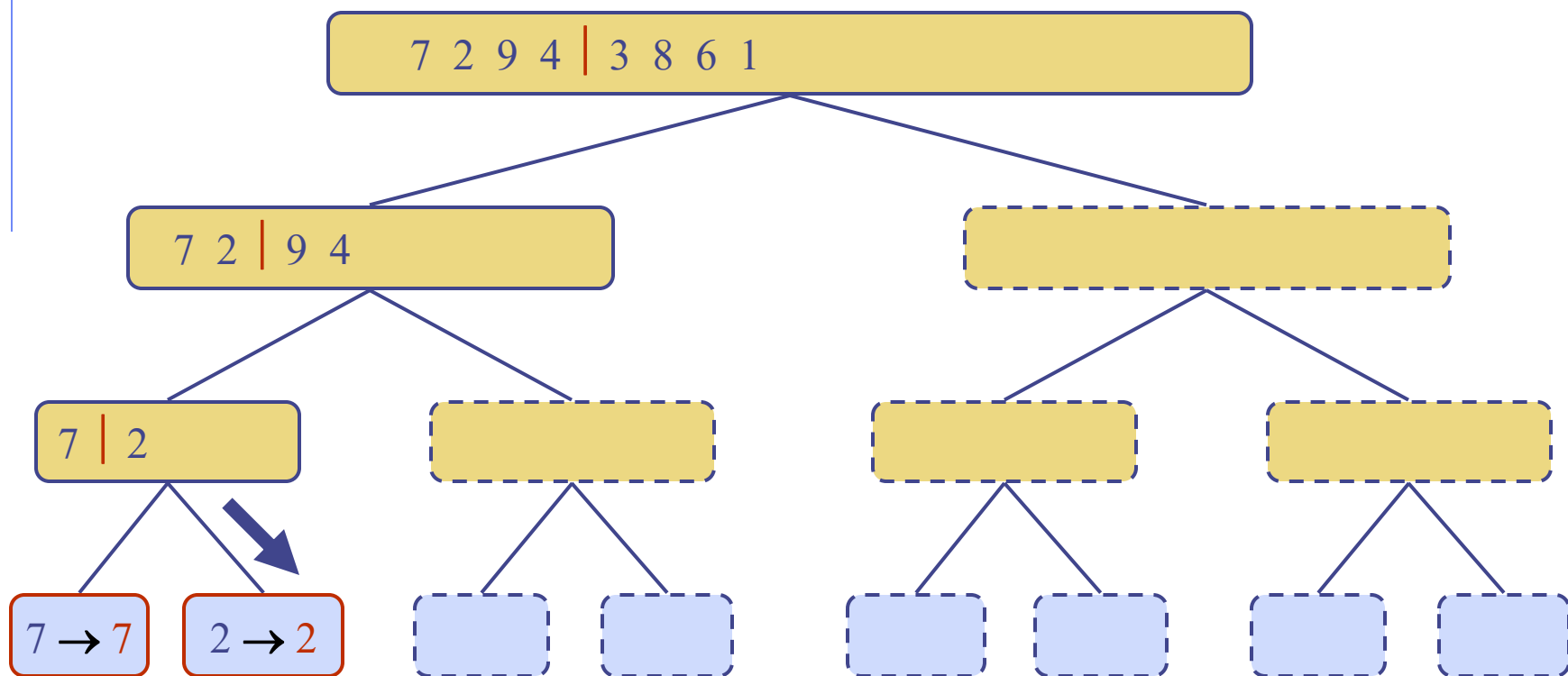
归并排序例

- 递归调用，出现基本情况



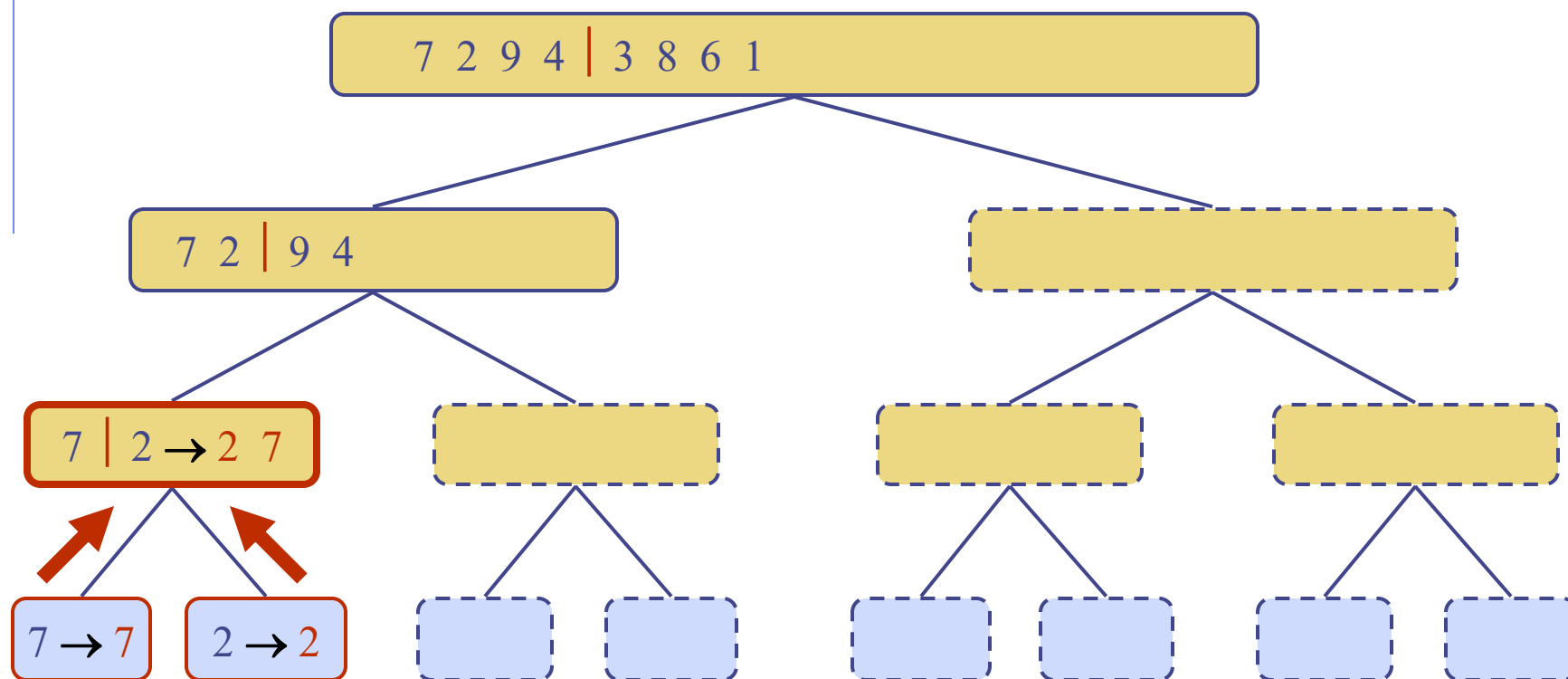
归并排序例

- 递归调用，出现基本情况



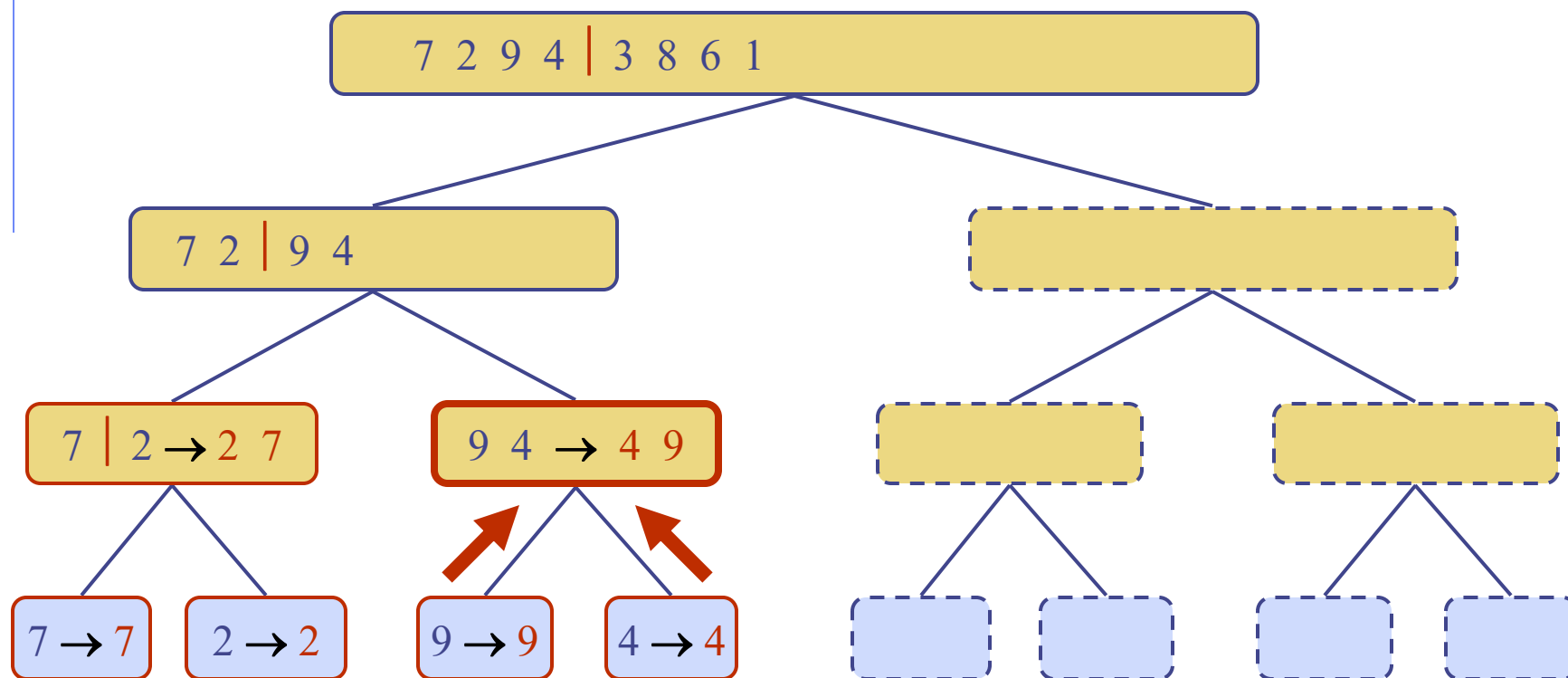
归并排序例

- 合并



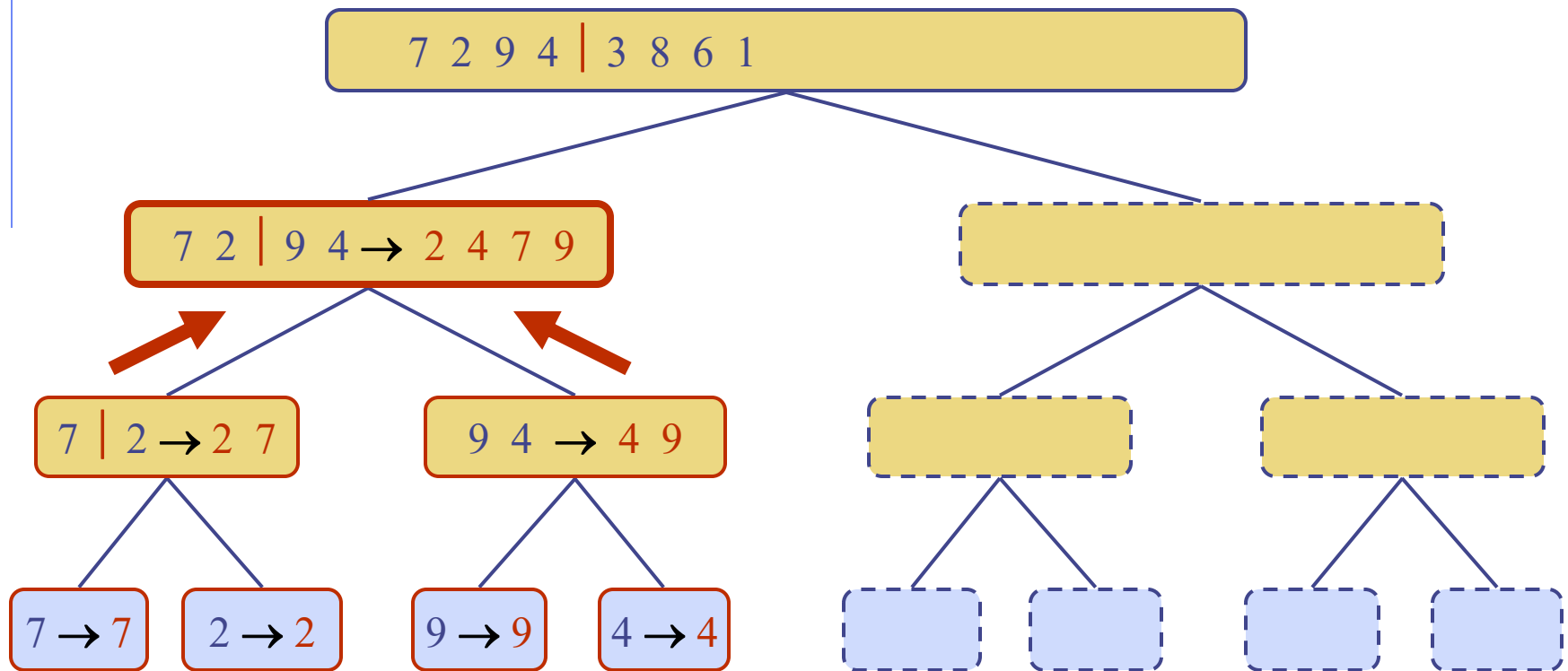
归并排序例

- 递归调用，…，出现基本情况，合并



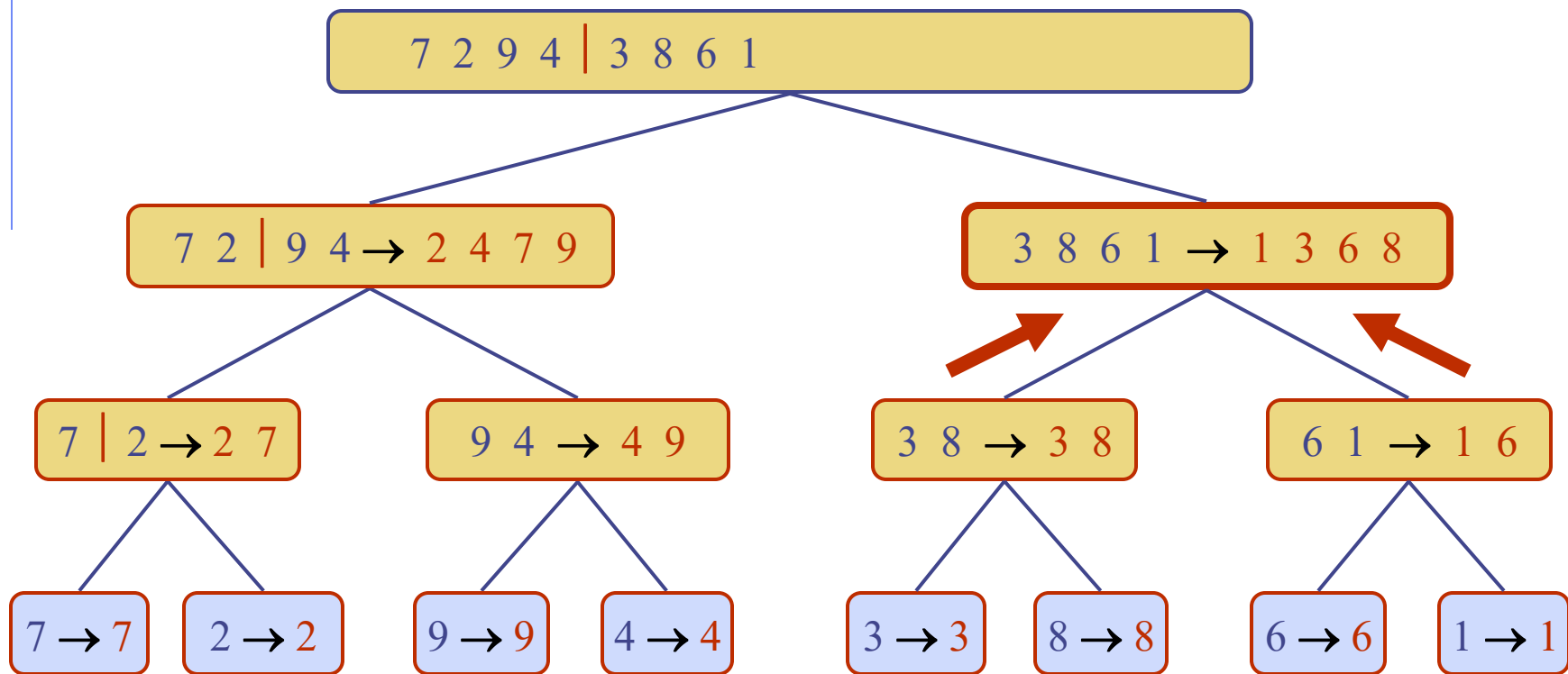
归并排序例

- 合并



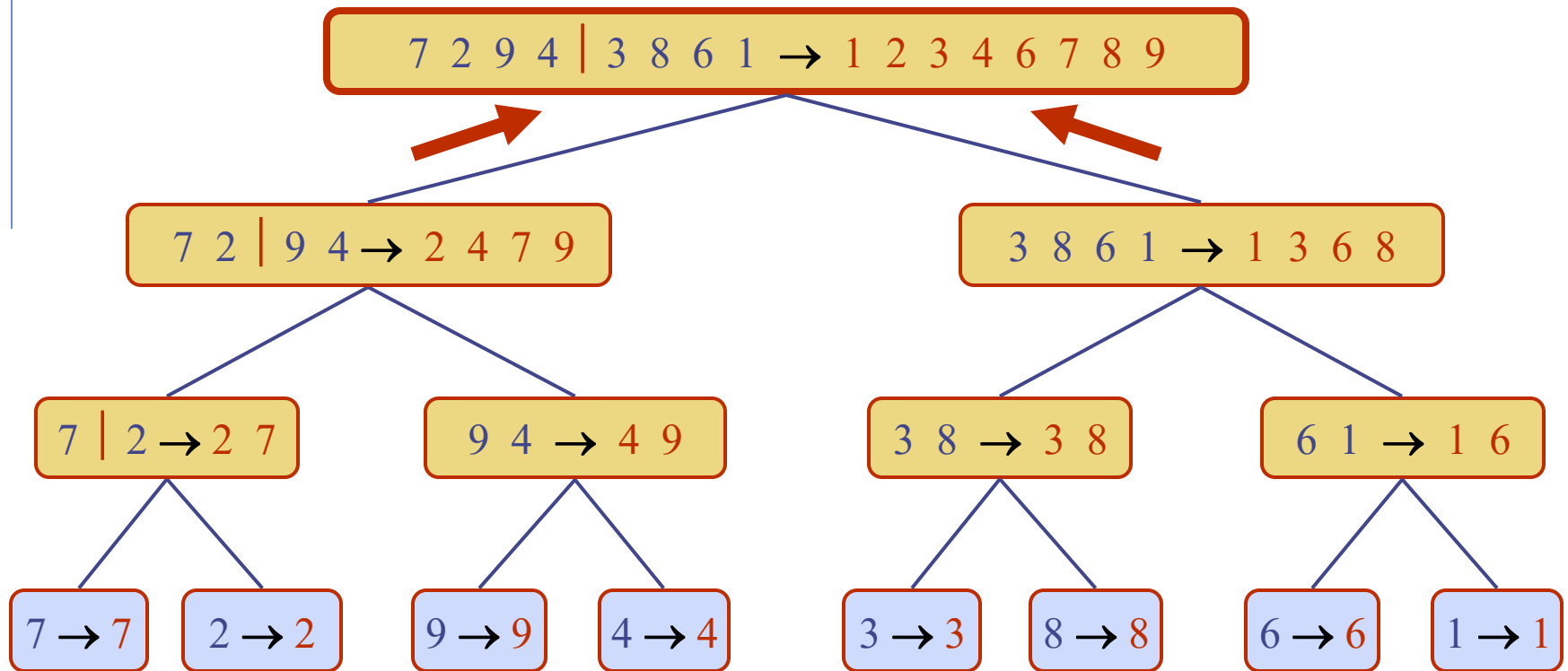
归并排序例

- 递归调用, ..., 合并, ...



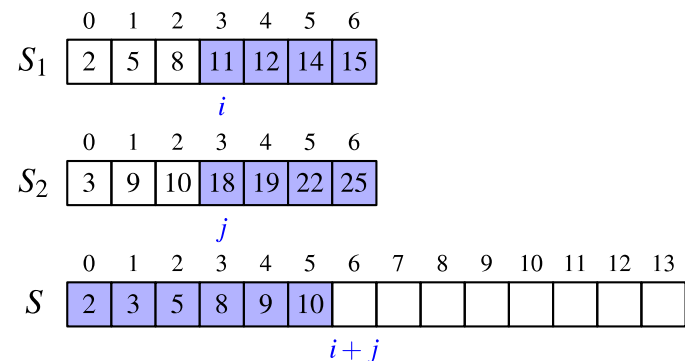
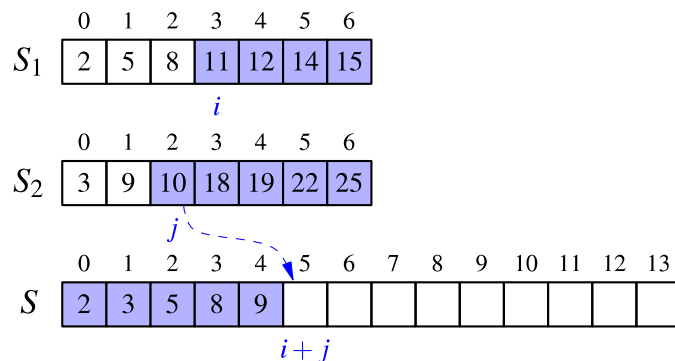
归并排序例

· 合并



归并排序的实现

```
def merge(S1, S2, S):
    """Merge two sorted Python lists S1 and S2 into properly sized list S."""
    i = j = 0
    while i + j < len(S):
        if j == len(S2) or (i < len(S1) and S1[i] < S2[j]):
            S[i+j] = S1[i]          # copy ith element of S1 as next item of S
            i += 1
        else:
            S[i+j] = S2[j]          # copy jth element of S2 as next item of S
            j += 1
```



归并排序的实现

```
def merge_sort(S):
    """Sort the elements of Python list S using the merge-sort algorithm."""
    n = len(S)
    if n < 2:
        return                # list is already sorted
    # divide
    mid = n // 2
    S1 = S[0:mid]             # copy of first half
    S2 = S[mid:n]             # copy of second half
    # conquer (with recursion)
    merge_sort(S1)            # sort copy of first half
    merge_sort(S2)            # sort copy of second half
    # merge results
    merge(S1, S2, S)          # merge sorted halves back into S
```


归并排序的性能分析

- 归并排序树的高度 h 为 $O(\log n)$
- 在深度为 i 的一层所需的时间为 $O(n)$
 - 我们对长度为 $n/2^i$ 的 2^i 个序列进行合并
 - 并进行 2^{i+1} 次递归调用
- 因此，归并排序的运行时间为 $O(n \log n)$

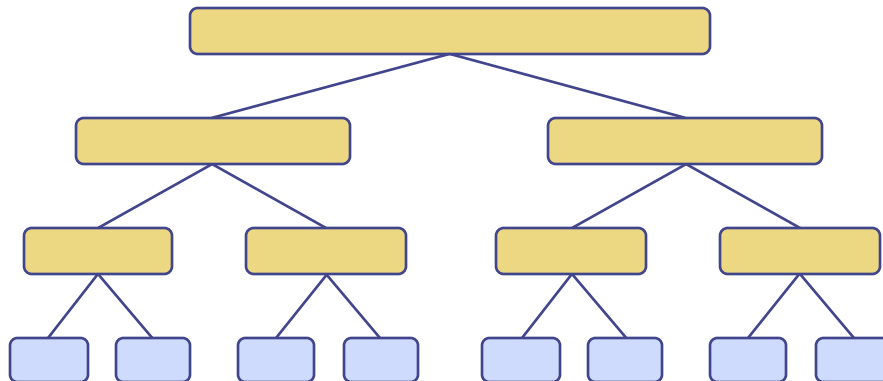
depth	#seqs	size
-------	-------	------

0	1	n
---	---	-----

1	2	$n/2$
---	---	-------

i	2^i	$n/2^i$
-----	-------	---------

...	...	...
-----	-----	-----

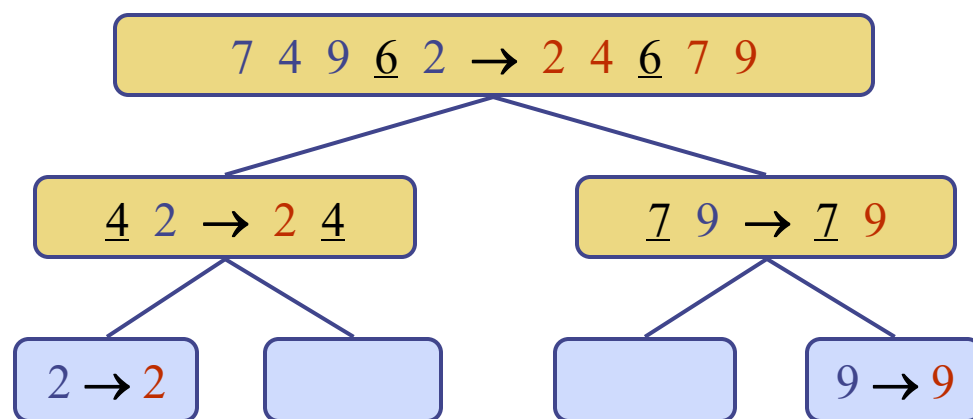




排序

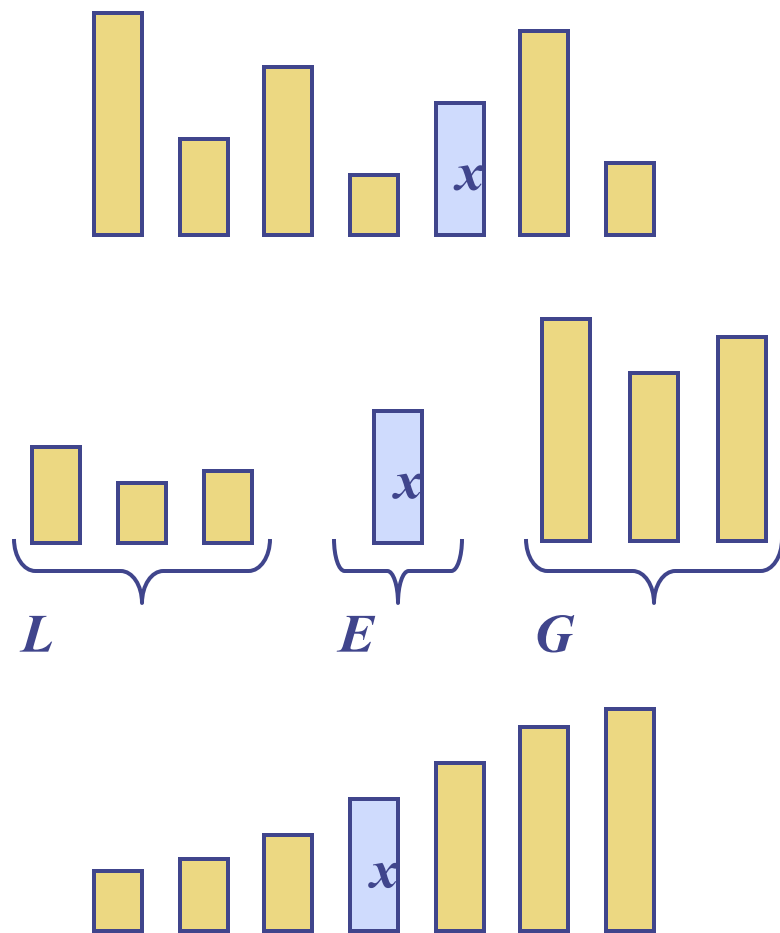
- 归并排序
- 快速排序
- 排序下界
- 桶排序和基数排序

快速排序

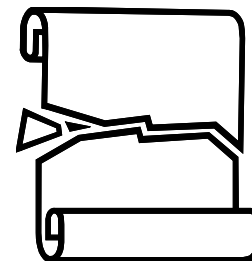


快速排序

- 快速排序 (quick-sort) 是一种基于分治法的随机排序算法，在序列 S 上其执行过程如下：
 - 分解 (Divide)：随机选取一个元素 x ，此元素被称为基准值 (pivot)，将序列分割为三部分：
 - L 存储小于 x 的元素
 - E 存储等于 x 的元素
 - G 存储大于 x 的元素
 - 解决子问题 (Conquer)：递归地对 L 和 G 排序
 - 合并 (Combine)：合并 L , E 和 G



分割



- 我们采用如下方式对一个序列 S 进行分割：
 - 我们将 S 中的每一个元素 y 移出序列
 - 根据 y 与基准值 x 的比较结果，将其插入 L , E 或 G
- 每一次插入或删除操作是在序列的尾或头进行的，因此其运行时间复杂度为 $O(1)$
- 由此可得，快速排序中每次分割操作的时间复杂度为 $O(n)$

Algorithm *partition*(S, p)

Input sequence S , position p of pivot
Output subsequences L, E, G of the elements of S less than, equal to, or greater than the pivot, resp.

$L, E, G \leftarrow$ empty sequences

$x \leftarrow S.remove(p)$

while $\neg S.isEmpty()$

$y \leftarrow S.remove(S.first())$

if $y < x$

$L.addLast(y)$

else if $y = x$

$E.addLast(y)$

else $\{ y > x \}$

$G.addLast(y)$

return L, E, G

快速排序的实现

```
def quick_sort(S):  
    """Sort the elements of queue S using the quick-sort algorithm."""  
    n = len(S)  
    if n < 2:  
        return # list is already sorted  
    # divide # using first as arbitrary pivot  
    p = S.first()  
    L = LinkedQueue()  
    E = LinkedQueue()  
    G = LinkedQueue()  
    while not S.is_empty(): # divide S into L, E, and G  
        if S.first() < p:  
            L.enqueue(S.dequeue())  
        elif p < S.first():  
            G.enqueue(S.dequeue())  
        else: # S.first() must equal pivot  
            E.enqueue(S.dequeue())
```

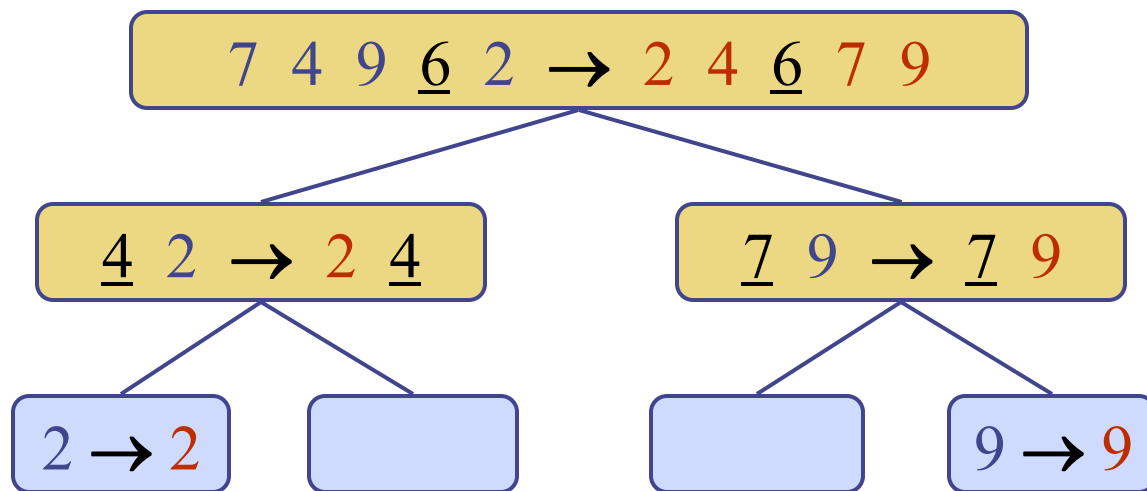
快速排序的实现

```
# conquer (with recursion)
quick_sort(L)
quick_sort(G)
# concatenate results
while not L.is_empty():
    S.enqueue(L.dequeue())
while not E.is_empty():
    S.enqueue(E.dequeue())
while not G.is_empty():
    S.enqueue(G.dequeue())
```

```
# sort elements less than p
# sort elements greater than p
```

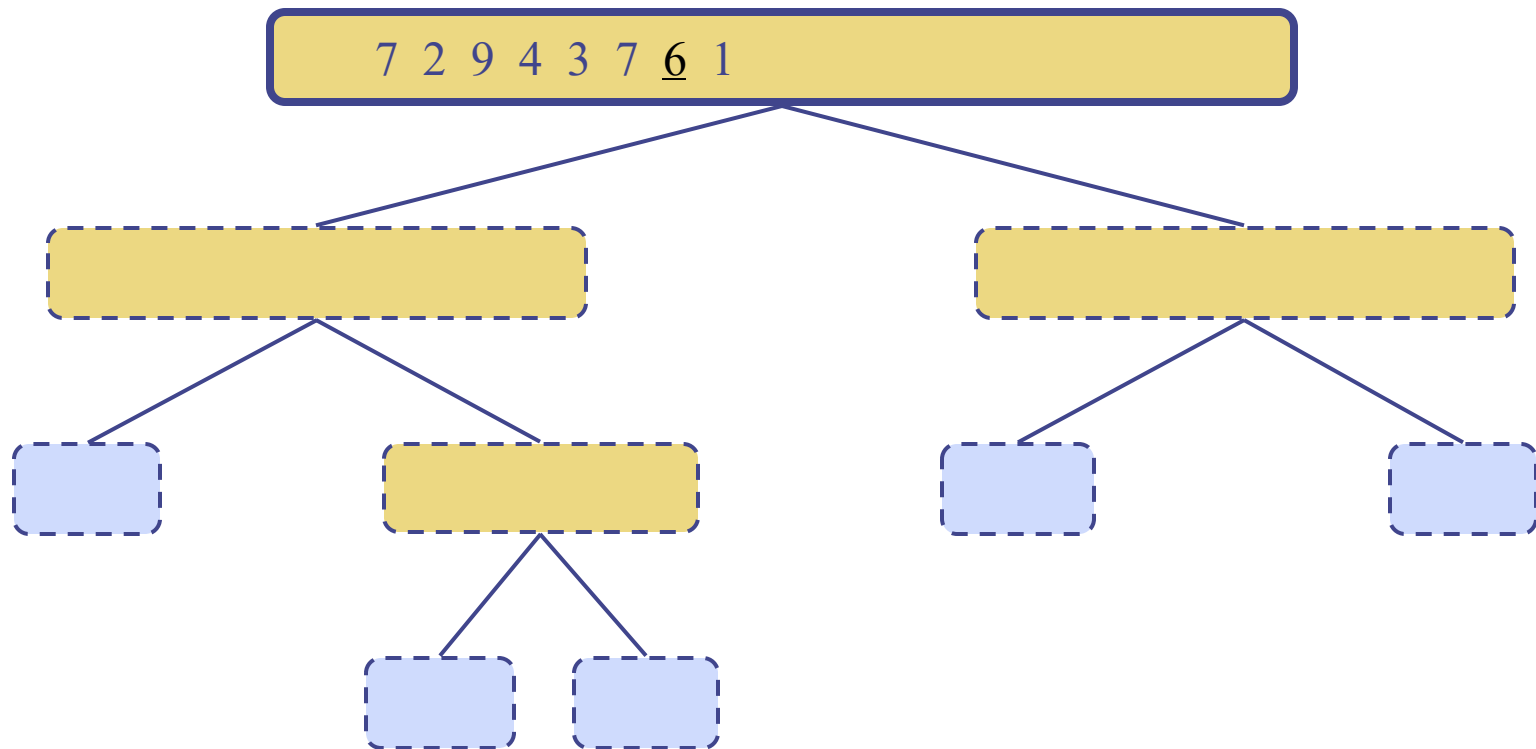

快速排序树

- 与归并排序一样，快速排序的执行也可以用一棵二叉树表示，称为**快速排序树** (quick-sort tree)
 - 每个节点表示一次快速排序的递归调用，并存储如下信息：
 - 执行排序前的无序序列及基准值
 - 执行排序后的有序序列
 - 根节点表示首次调用快速排序
 - 叶子节点表示在长度为0或1的序列上的调用



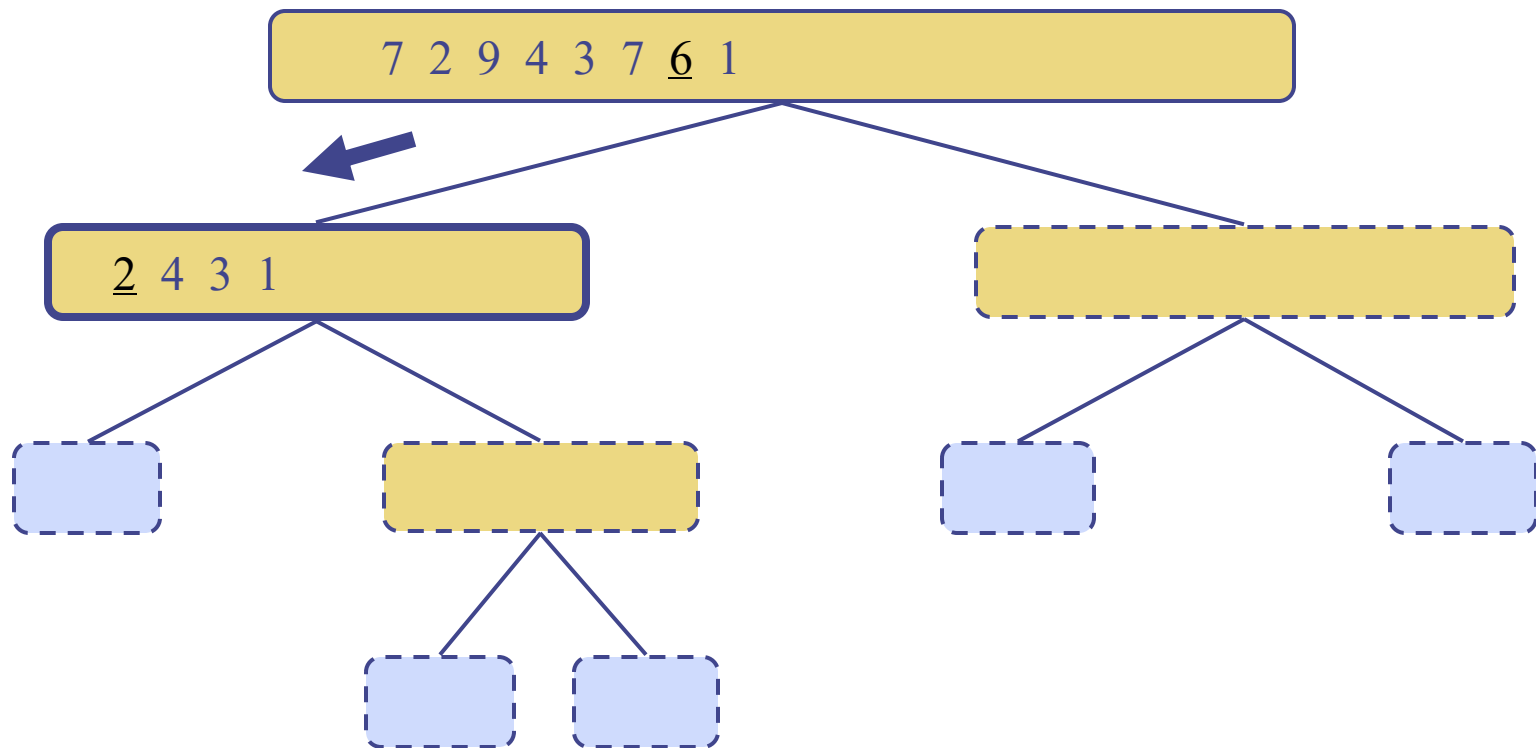
快速排序例

- 选取基准值



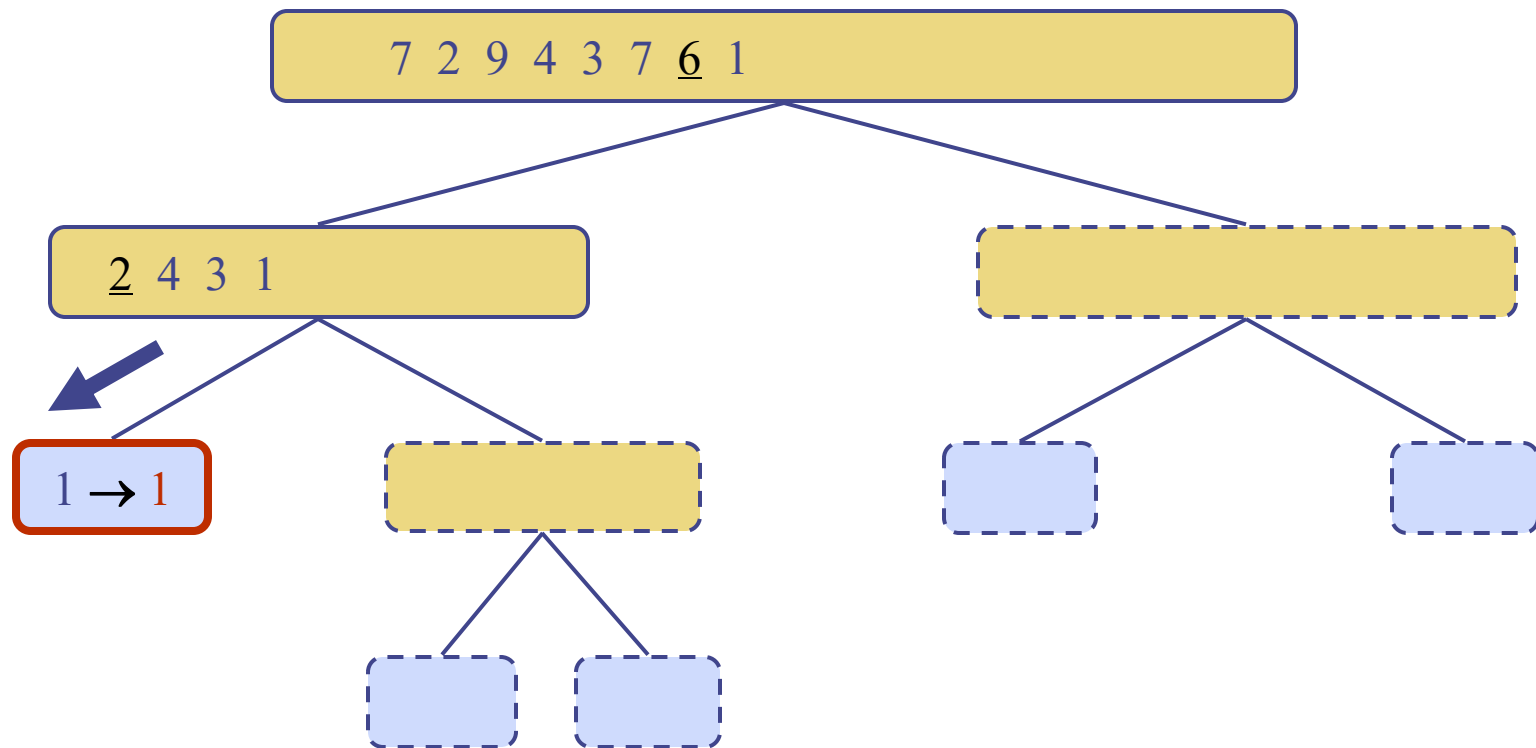
快速排序例

- 分割，递归调用，再选取基准值



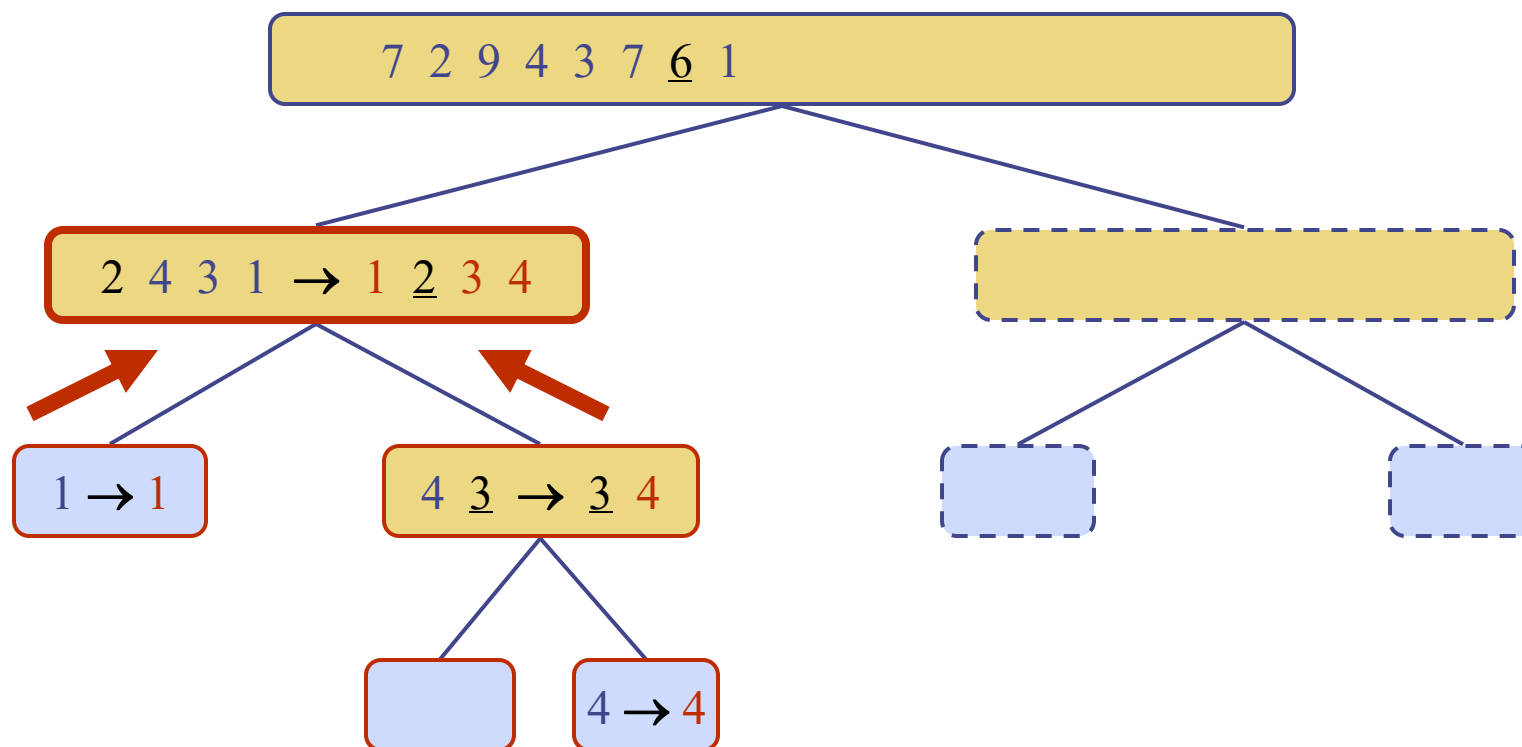
快速排序例

- 分割，递归调用，出现基本情况



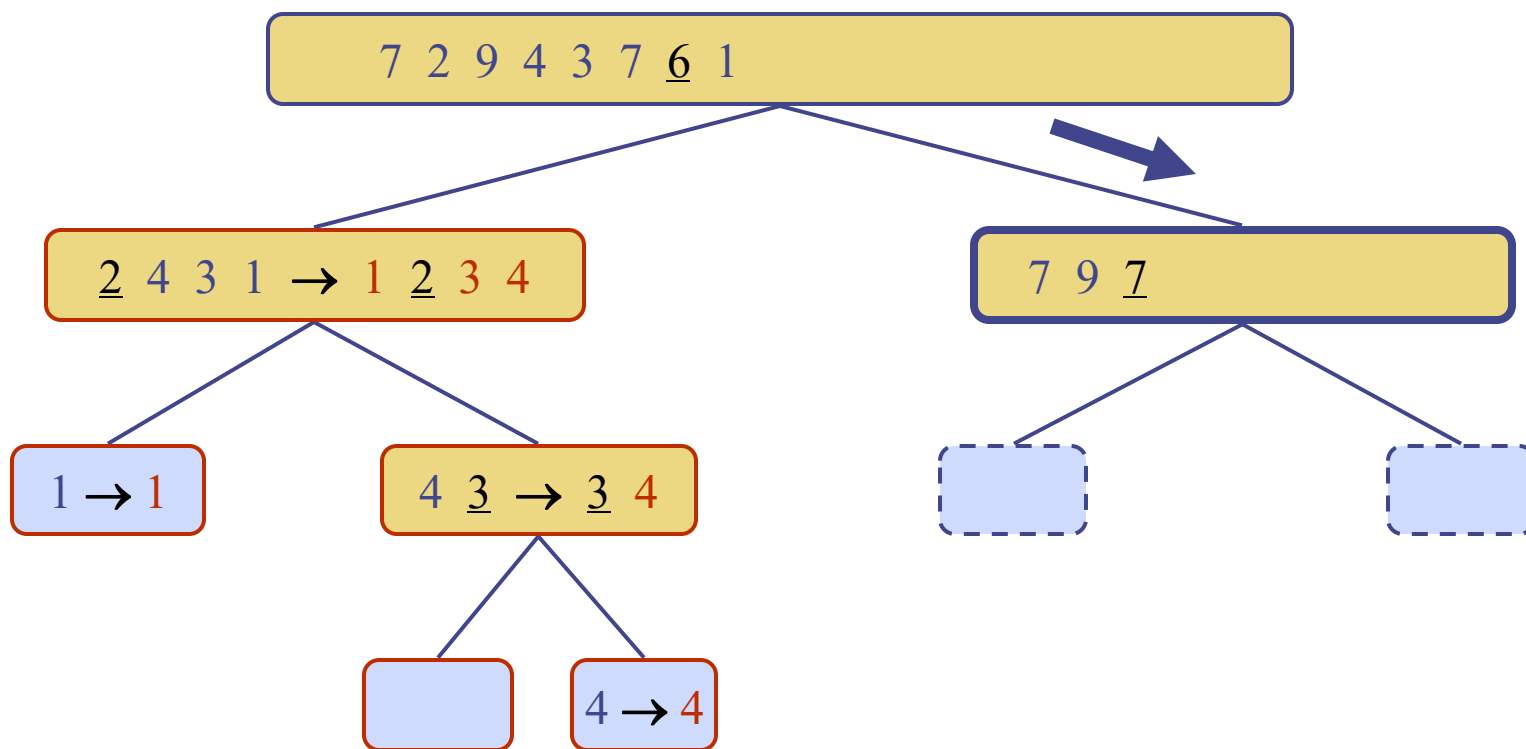
快速排序例

- 递归调用，…，出现基本情况，合并



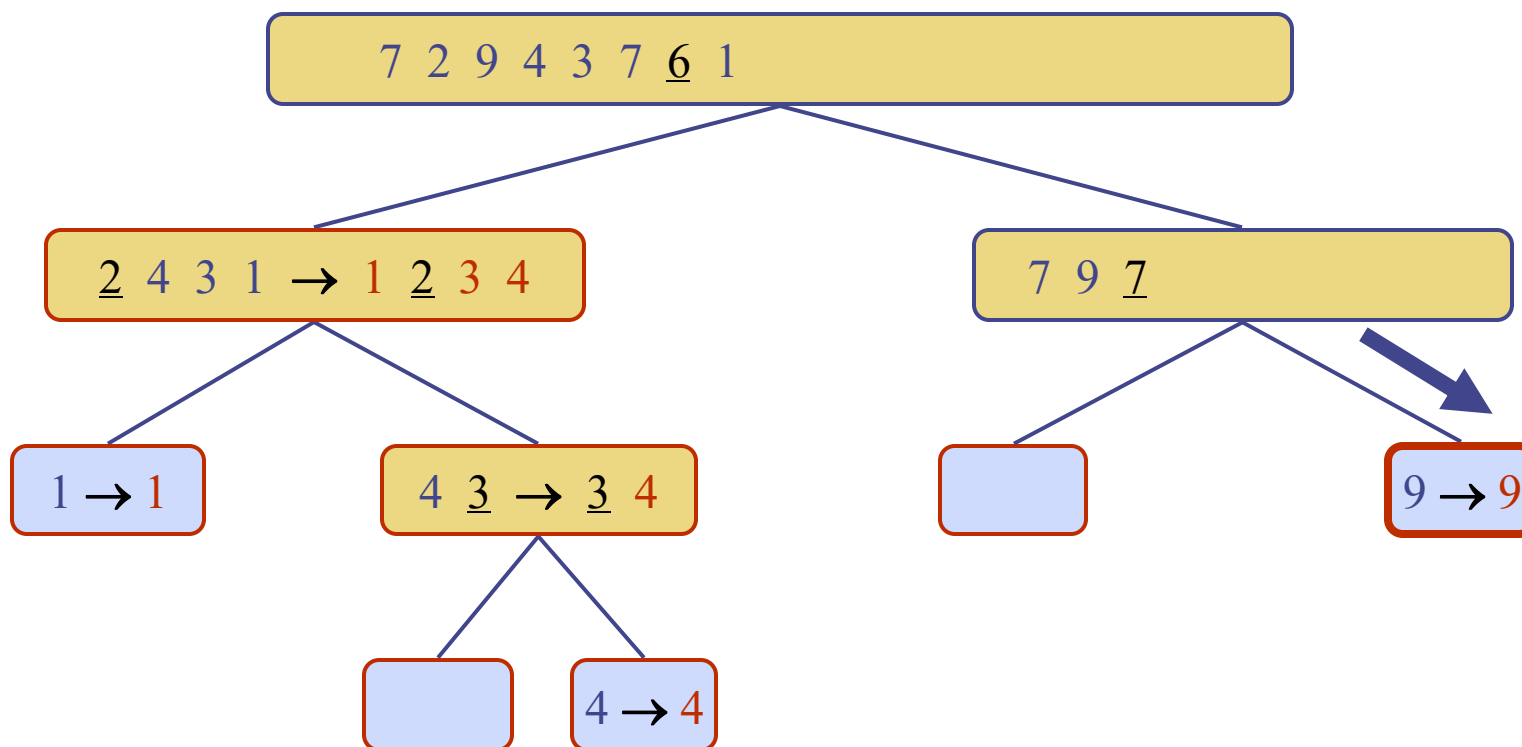
快速排序例

- 递归调用，选取基准值



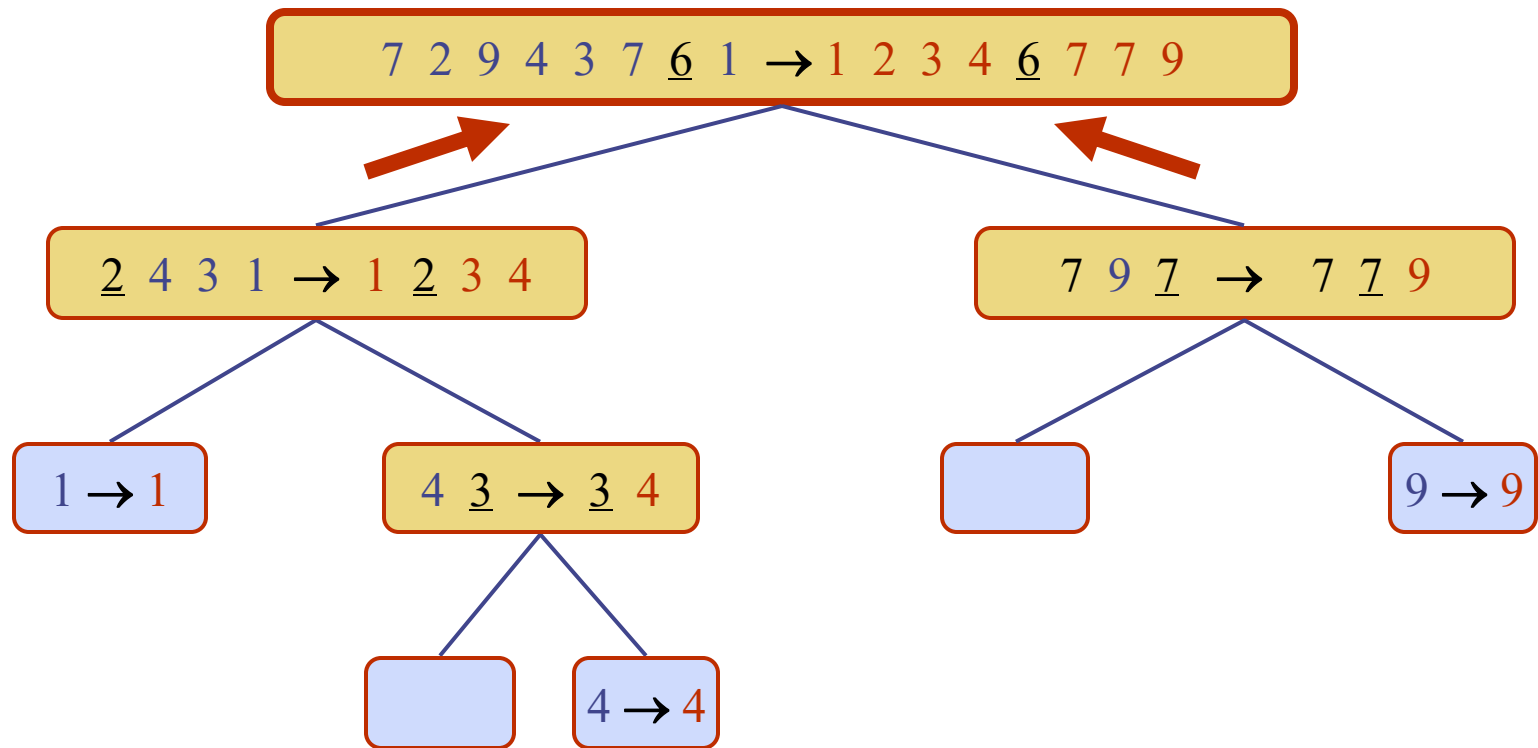
快速排序例

- 分割，递归调用，出现基本情况



快速排序例

· 合并



快速排序的性能分析

- 与归并排序不同，最坏情况下，快速排序效率不甚理想
- 最坏情况下，快速排序每一次选取的**基准值**都恰好为序列中唯一的**最大值或最小值**
 - L 和 G 中一个长度为 $n - 1$ ，而另一个长度为 0
 - 快速排序树高度为 $n - 1$
 - 运行时间与 $n + (n - 1) + \dots + 2 + 1$ 成正比
- 因此，最坏情况下，快速排序的运行时间为 $O(n^2)$

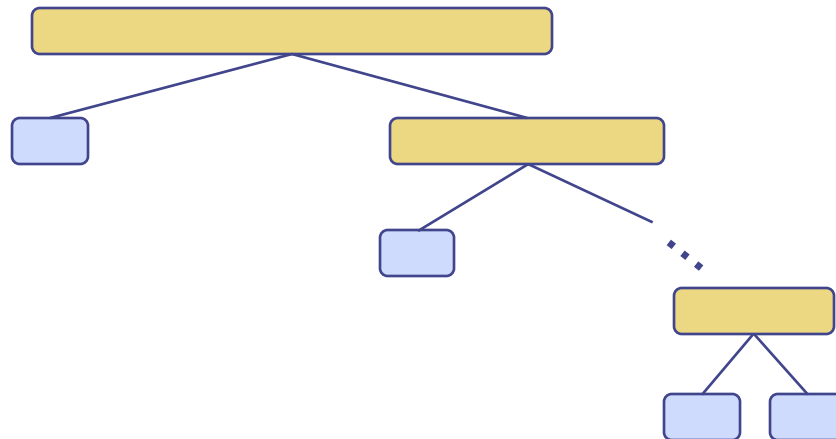
depth time

0 n

1 $n - 1$

...

$n - 1$ 1

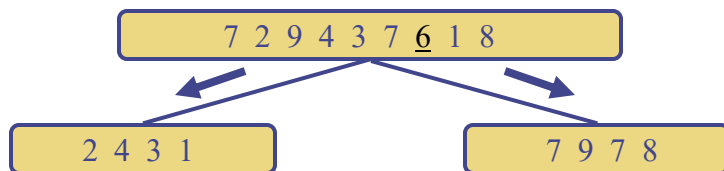


快速排序的性能分析

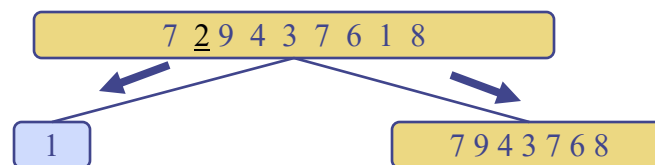
- 我们分析快速排序时，通常假设基准值总是能将序列以一种合理的方式进行分割，即最坏情况很少出现
- 实际上，我们可以通过**随机快速排序**使得快速排序运行的时间接近最好情况
- 随机快速排序中，我们每一次进行排序时均**随机选择一个元素作为基准值**
- 在此情况下，我们可以计算快速排序的**期望运行时间**

快速排序的性能分析

- 考虑在长为 n 的序列上的一次随机快速排序的调用：
 - “好” 的调用： L 和 G 的长度均不大于 $3n/4$
 - “坏” 的调用： L 和 G 中的一个长度大于 $3n/4$



“好” 的调用



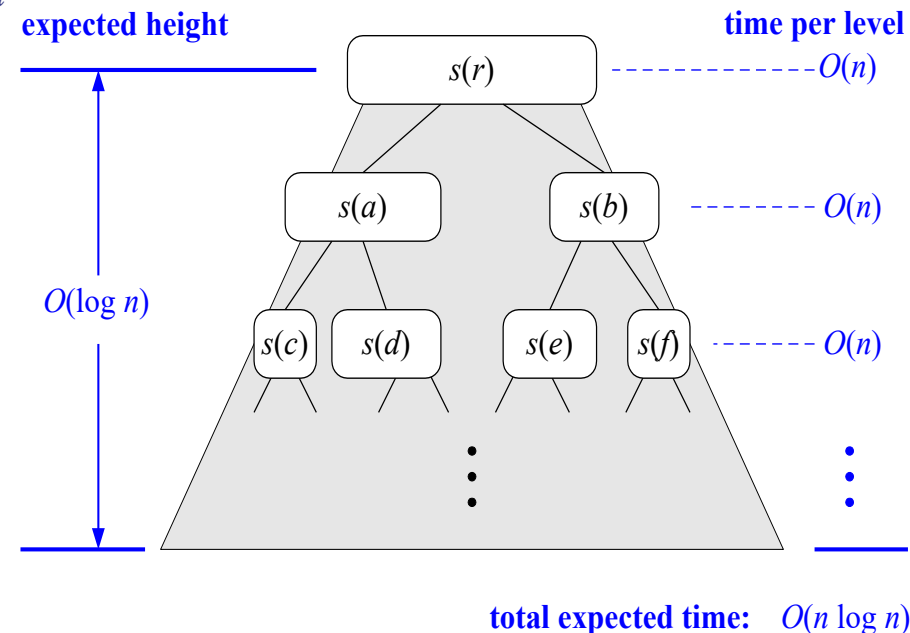
“坏” 的调用

- 一次调用为 “好” 的概率为 $1/2$
 - $1/2$ 的基准值可能产生 “好” 的调用：



快速排序的性能分析

- 由概率论相关知识可知，当一个好的调用出现概率为 $1/2$ 时，为了得到一个好的调用，我们不得不进行的连续调用次数的期望为2
- 对于深度为 i 的节点，在期望意义下，有：
 - 其 $i/2$ 的祖先节点处产生了好的调用
 - 当前节点处序列的长度至多为 $(3/4)^{i/2}n$
- 因此，我们可以得到：
 - 深度为 $2\log_{4/3}n$ 的节点的序列长度期望为1
 - 快速排序树的高度期望为 $O(\log n)$
- 在同一深度上进行操作的期望时间为 $O(n)$
- 因此，快速排序的期望运行时间为 $O(n \log n)$





就地快速排序

- 快速排序有就地实现的版本
- 在分割这一步骤，我们通过元素的交换，对序列重新进行排布，使得：
 - 比基准值小的元素排在第 h 位之前
 - 与基准值相等的元素排在第 h 位到第 k 位之间
 - 比基准值大的元素排在第 k 位之后
- 递归调用在如下两部分序列上进行：
 - 第 h 位之前的序列
 - 第 k 位之后的序列

Algorithm *inPlaceQuickSort*(S, l, r)

Input sequence S , ranks l and r

Output sequence S with the elements of rank between l and r rearranged in increasing order

if $l \geq r$

return

$x \leftarrow$ a random element between rank l and r

$(h, k) \leftarrow \text{inPlacePartition}(x)$

inPlaceQuickSort($S, l, h - 1$)

inPlaceQuickSort($S, k + 1, r$)

就地分割



- 在就地分割这一步骤，我们使用两个索引，将序列 S 分割为 L 和 EUG

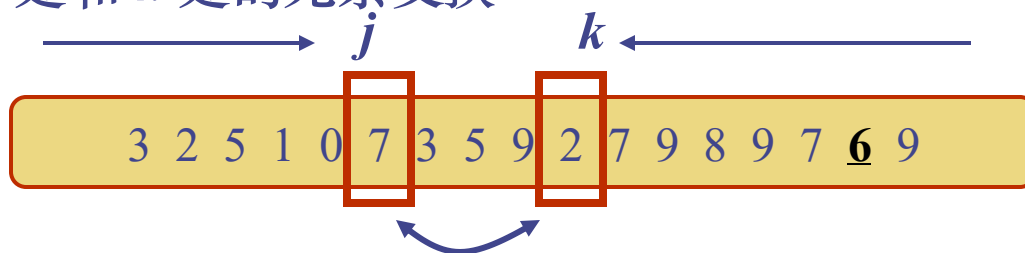
j

k

3 2 5 1 0 7 3 5 9 2 7 9 8 9 7 6 9

(pivot = 6)

- 不断重复如下步骤，直到 j 与 k 相交：
 - 从左向右移动 j ，直到找到一个大于等于 x 的元素
 - 从右向左移动 k ，直到找到一个小于 x 的元素
 - 将 j 处和 k 处的元素交换



一种就地快速排序的实现

```
def inplace_quick_sort(S, a, b):  
    """Sort the list from S[a] to S[b] inclusive using the quick-sort algorithm."""  
    if a >= b: return # range is trivially sorted  
    pivot = S[b] # last element of range is pivot  
    left = a # will scan rightward  
    right = b-1 # will scan leftward  
    while left <= right:  
        # scan until reaching value equal or larger than pivot (or right marker)  
        while left <= right and S[left] < pivot:  
            left += 1  
        # scan until reaching value equal or smaller than pivot (or left marker)  
        while left <= right and pivot < S[right]:  
            right -= 1  
        if left <= right: # scans did not strictly cross  
            S[left], S[right] = S[right], S[left] # swap values  
            left, right = left + 1, right - 1 # shrink range  
    # put pivot into its final place (currently marked by left index)  
    S[left], S[b] = S[b], S[left]  
    # make recursive calls  
    inplace_quick_sort(S, a, left - 1)  
    inplace_quick_sort(S, left + 1, b)
```

基于比较的排序算法

算法	时间复杂度	特点
选择排序	$O(n^2)$	• 可就地排序 • 慢（小规模数据上尚可）
插入排序	$O(n^2)$	• 可就地排序 • 慢（小规模数据上表现佳）
堆排序	$O(n \log n)$	• 可就地排序 • 快（适合中、大型规模的数据）
归并排序	$O(n \log n)$	• 可顺序访问数据并同时排序 • 快（适合超大型规模的数据）
快速排序	$O(n \log n)$ (期望值)	• 可就地排序，随机化基准值 • 快（适合中、大型规模的数据）

课堂练习

- 合并 k 个升序链表
 - 给你一个链表数组，每个链表都已经按升序排列
 - 请你将所有链表合并到一个升序链表中，返回合并后的链表
 - 如何做效率最高？时间复杂度为？

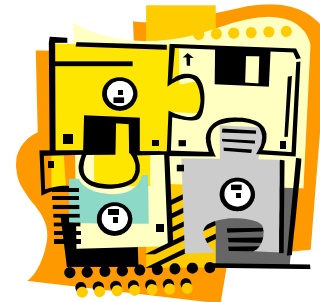


排序

- 归并排序
- 快速排序
- 排序下界
- 桶排序和基数排序

排序下界





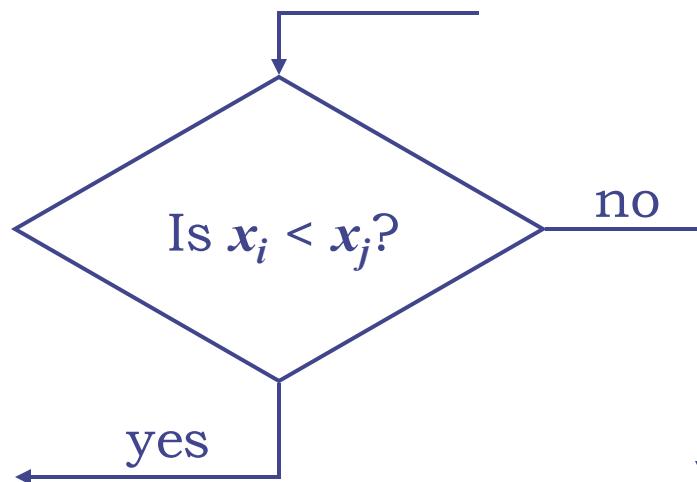
排序下界

- 我们可以注意到，相对高效的算法的时间复杂度均为 $O(n \log n)$
 - 如堆排序、快速排序、归并排序
- 是否有更快的排序方法？**排序时间**是否有下界？
- 结论：对基于比较的排序算法， $O(n \log n)$ 已经是最好的结果
 - Why?

基于比较的排序算法

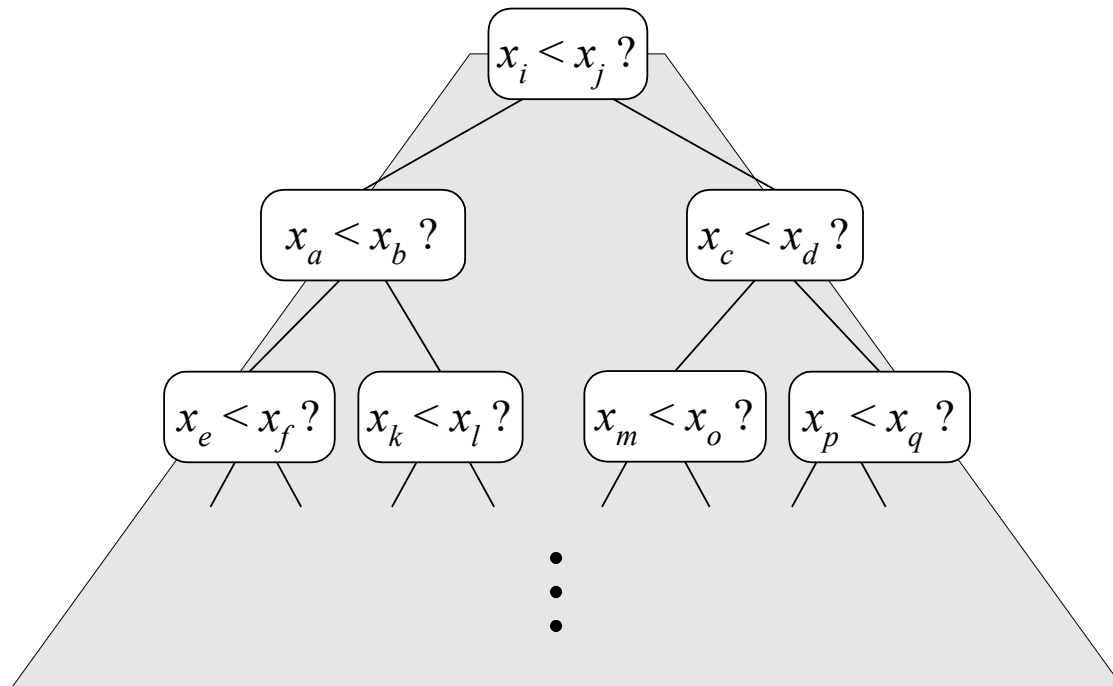


- 许多排序算法是基于比较的
 - 这些算法通过对不同元素间的**比较**操作进行排序
 - 例：选择排序、插入排序、冒泡排序、堆排序、归并排序、快速排序……
- 我们来分析基于比较的排序算法的**运行时间下界**



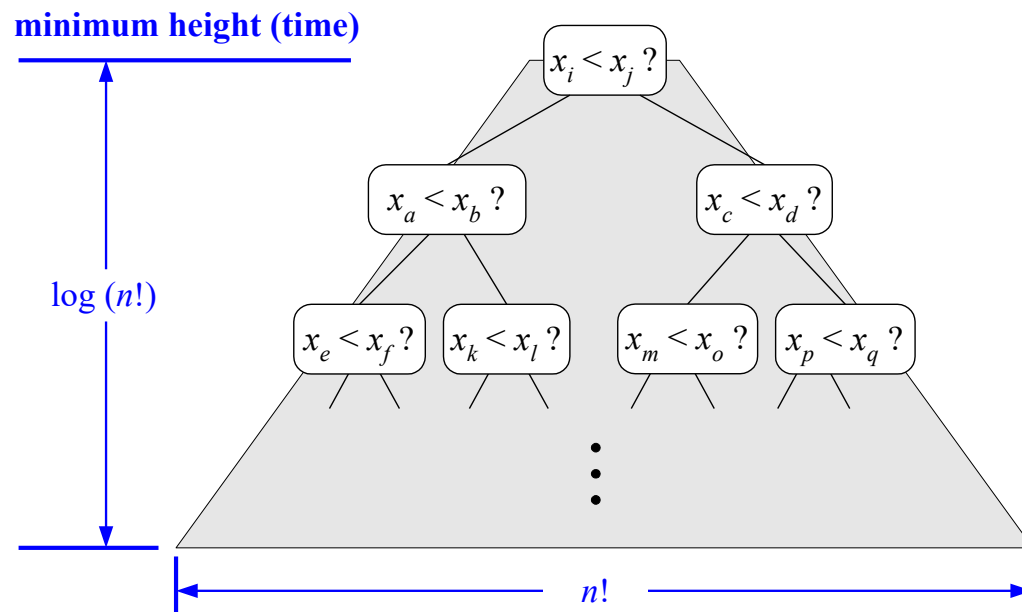
计算比较次数

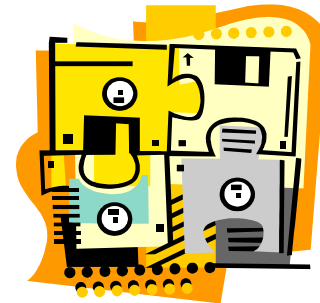
- 由于我们讨论的是排序下界，因此只计算进行的**比较操作**的次数
- 算法的每次可能的运行均可对应于一棵**决策树** (decision tree)
- 在每个节点处，根据比较的结果，排序算法进行不同的操作



决策树的高度

- 决策树的高度决定了排序算法运行时间的下界
- 对每一种可能的序列的排列 (permutation)，必须有唯一的一个叶子节点与之对应
 - 否则，一个排列为 $\cdots x_i \cdots x_j \cdots$ 的序列与一个排列为 $\cdots x_j \cdots x_i \cdots$ 的序列的排序方法是完全一致的，这将导致其中的一个排序结果是错误的
- 叶子节点的数目为 n 个元素的排列数目，即 $n!$ ，因此决策树高度至少为 $\log(n!)$





计算排序下界

- 进一步简化结论：

$$\log(n!) \geq \log\left(\frac{n}{2}\right)^{\frac{n}{2}} = (n/2) \log(n/2).$$

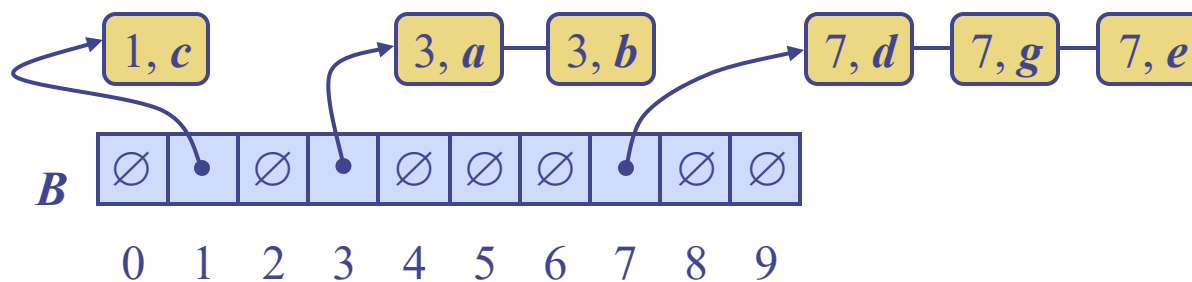
- 因此，任何**基于比较**的排序算法的运行时间下界为 $\Omega(n \log n)$
- 是否有其他类型的排序算法有更高的效率？



排序

- 归并排序
- 快速排序
- 排序下界
- 桶排序和基数排序

桶排序和基数排序





桶排序

- 在序列元素有特殊形式的情况下，可以采用桶排序 (bucket-sort)
- 假设序列 S 中有 n 个键值对，且键为 $[0, N-1]$ 内的整数
- 桶排序将键作为索引，将原序列中的元素插入一个辅助数组 B 中
 - 阶段1：将序列 S 中的元素插入数组 B 中，具体方式为将元素 (k, v) 放入“桶” $B[k]$ 中
 - 阶段2：按序枚举 $B[0], B[1], \dots, B[N-1]$ ，将桶内的元素依次放回序列 S 中
- 运行时间分析：
 - 阶段1需要 $O(n)$ 的时间
 - 阶段2需要 $O(N)$ 的时间
 - 桶排序的运行时间为 $O(n + N)$



桶排序

Algorithm *bucketSort*(S):

Input: Sequence S of entries with integer keys in the range $[0, N - 1]$

Output: Sequence S sorted in nondecreasing order of the keys

let B be an array of N sequences, each of which is initially empty

for each entry e in S **do**

k = the key of e

 remove e from S and insert e at the end of bucket $B[k]$

for $i = 0$ to $N - 1$ **do**

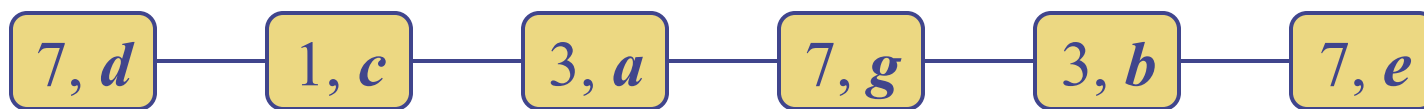
for each entry e in $B[i]$ **do**

 remove e from $B[i]$ and insert e at the end of S

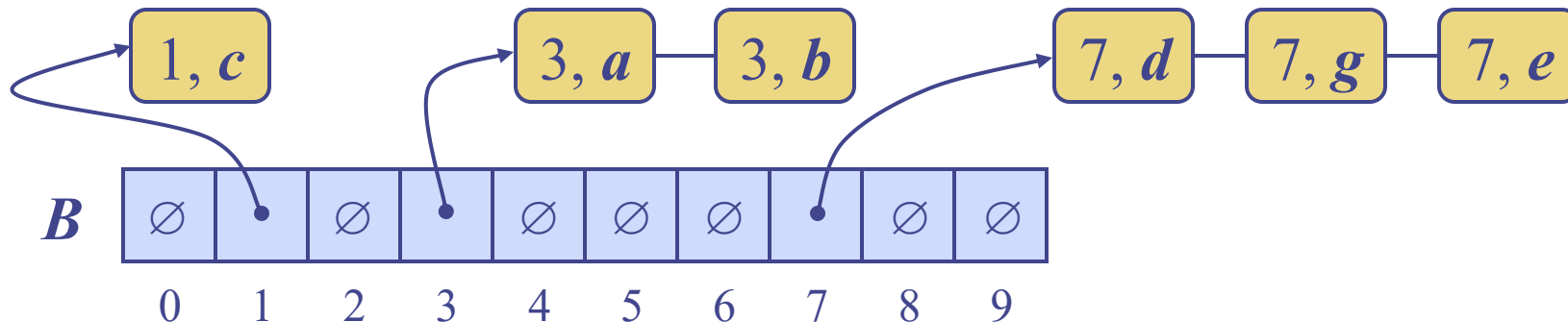
桶排序例



- 键范围为 $[0, 9]$



阶段1



阶段2



桶排序的特点及性质

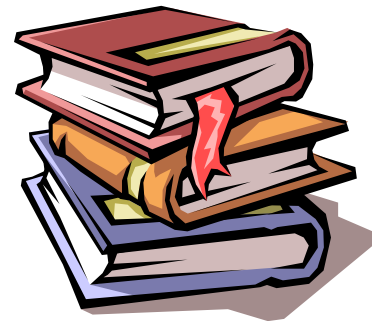


- 键的特点与性质
 - 键被用作插入辅助数组的索引，因此对键的类型有所限制
 - 不需要使用额外的比较器（comparator）进行比较操作
- 稳定排序（stable sorting）性质
 - 如果两个键相同的元素在排序前的顺序与排序后的顺序一致，则称此排序方法为稳定的
 - 桶排序是一种稳定排序算法
 - 堆排序、快速排序、归并排序是稳定排序吗？

桶排序的拓展应用



- 桶排序可以应用在键有特殊性质的情况下：
 - 键为 $[a, b]$ 范围内的整数
 - 将元素 (k, v) 放入桶 $B[k - a]$
 - 键为字符串，且在一个大小固定的、元素有序的集合 D 中（如国内34个省级行政区划）
 -



字典序排序

- 一个 d 元组为一个由 d 个键构成的序列 $(k_1, k_2, \dots, k_d)$, 其中键 k_i 被称为此 d 元组的第 i 个维度
- 例如, 三维空间中一个点的笛卡尔坐标为一个三元组
- 两个 d 元组的字典序顺序被定义为:

$$(x_1, x_2, \dots, x_d) < (y_1, y_2, \dots, y_d)$$



$$x_1 < y_1 \vee x_1 = y_1 \wedge (x_2, \dots, x_d) < (y_2, \dots, y_d)$$

- 即 d 元组的比较过程为先比较第1维元素, 如相等则再比较第2维,

字典序排序

- 我们可以设计一个进行字典序排序的方法
- 假设对第 i 维, 我们使用比较器 C_i 对其进行排序
- 假设我们找到了使用任意比较器 C 进行**稳定排序**的方法 $stableSort(S, C)$
- 字典序排序通过在对 d 元组的所有维度上依次进行稳定排序 $stableSort$ 而完成
- 字典序排序的运行时间为 $O(dT(n))$, 其中 $T(n)$ 为稳定排序 $stableSort$ 的运行时间

Algorithm *lexicographicSort(S)*

Input sequence S of d -tuples

Output sequence S sorted in lexicographic order

for $i \leftarrow d$ **down to** 1

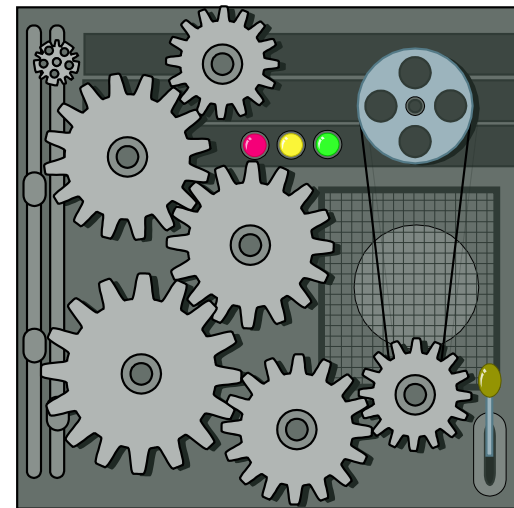
stableSort(S, C_i)

例:

(7, 4, 6) (5, 1, 5) (2, 4, 6) (2, 1, 4) (3, 2, 4)
 (2, 1, 4) (3, 2, 4) (5, 1, 5) (7, 4, 6) (2, 4, 6)
 (2, 1, 4) (5, 1, 5) (3, 2, 4) (7, 4, 6) (2, 4, 6)
 (2, 1, 4) (2, 4, 6) (3, 2, 4) (5, 1, 5) (7, 4, 6)

基数排序

- 基数排序 (radix-sort) 是一种特殊的字典序排序方式，它使用桶排序作为在每一维度上进行稳定排序的方式
- 基数排序适用于每一维度上均为 $[0, N-1]$ 范围内的整数的 d 元组
- 基数排序的运行时间为 $O(d(n + N))$



Algorithm *radixSort*(S, N)

Input sequence S of d -tuples such that $(0, \dots, 0) \leq (x_1, \dots, x_d)$ and $(x_1, \dots, x_d) \leq (N-1, \dots, N-1)$ for each tuple $(x_1, \dots, x_d)$ in S

Output sequence S sorted in lexicographic order

for $i \leftarrow d$ **down to** 1
 bucketSort(S_i, N)

ANY
QUESTIONS
?

www.rekrutitn.com

ReKrutitn.com

Enjoy